人工智能與機器學習入門講義

根據 W3Schools AI／Machine Learning 教程
（https://www.w3schools.com/ai/）全面整理

涵蓋：AI 概念與歷史 ・ 數學與統計基礎 ・ 感知器與神經網絡 ・ 梯度下降 ・ JavaScript／TensorFlow.js 實作 ・ 圖表視覺化

來源擷取日期：2026 年 10 月 14 日 ｜ 語言：繁體中文（香港）

【學習提示】 本講義共 14 節及一個附錄。第 0 節說明附件缺失、引用方式與方框標示慣例，請先閱讀。目錄中的每個條目均可點擊跳轉。

目錄

0. 前言與使用說明 9
0.1 本講義的目的與範圍 9
0.2 關於「附件資料」的重要說明 9
0.3 來源引用與標示慣例 9
0.4 原教程中無法於講義重現的元素 10
0.5 建議的閱讀順序 10
1. 人工智能、機器學習與深度學習的全貌 11
1.1 甚麼是人工智能（Artificial Intelligence）？ 11
1.2 人工智能的層級結構 12
1.3 窄人工智能（Narrow AI）與強人工智能（Strong AI） 12
1.3.1 窄人工智能（又稱弱人工智能 Weak AI） 12
1.3.2 強人工智能 13
1.4 機器學習與傳統編程的根本分別 13
1.5 神經網絡（Neural Networks）的基本概念 14
1.6 從感知器到深度神經網絡：層的概念 14
1.6.1 感知器（Perceptron） 14

1.6.2 神經網絡＝多層感知器.....14
1.6.3 深度神經網絡（Deep Neural Networks）.....15
1.7 深度學習（Deep Learning）.....15
1.8 「學習」的正式定義：Tom Mitchell 的 E–T–P 框架.....15
1.9 兩個說明「機器智能」的故事.....16
1.9.1 Giraffe 國際象棋程式（2015）.....16
1.9.2 「智能決策公式」.....16
2. 智能、計算與人工智能的歷史脈絡.....17
2.1 人類智能的起源.....17
2.1.1 詞語、數字與語言.....17
2.1.2 抽象思維（Abstract Thinking）.....17
2.1.3 甚麼是智能？.....17
2.1.4 人腦是否更優越？.....18
2.2 語言的歷史.....18
2.2.1 口語的起源.....18
2.2.2 認知發展的六個方面.....18
2.2.3 人類語言與電腦語言.....19
2.2.4 書面語言年表.....19
2.3 數字的歷史.....19
2.3.1 AI 關乎數字與計數.....20
2.3.2 巴比倫數字：六十進制（Base 60）.....20
2.3.3 十二進制（Dozenal, Base 12）.....21
2.3.4 羅馬數字：十進制（Base 10）.....21
2.3.5 數碼數字：二進制（Base 2）.....21
2.4 計算的歷史.....21
2.4.1 算盤（Abacus）.....22
2.4.2 類比計算機（Analog Computers）.....22
2.4.3 數碼計算機（Digital Computers）.....22

2.4.4 電子計算機的三個世代.....22
2.4.5 電腦速度.....22
2.5 機械人的歷史.....23
2.5.1 工業機械人.....23
2.5.2 AI 機械人.....24
2.6 人工智能大事年表.....24
2.6.1 為何是現在？（Why AI Now?）.....25
2.6.2 人與機器的對比及開放問題.....25
2.6.3 人工智能電影.....25
2.7 心智理論（Theory of Mind）與 AI 的未來階段.....27
2.7.1 甚麼是心智？認知科學與心智模型.....27
2.7.2 AI 能否成為人？四個階段.....27
2.8 人工智能與職業取代.....28
3. 機器學習的數學基礎：線性函數與線性代數.....30
3.1 機器學習涉及的數學分支與相關職業.....30
3.2 線性函數（Linear Functions）.....30
3.2.1 函數的概念.....30
3.2.2 線性方程與非線性方程.....31
3.2.3 線性迴歸與線性最小二乘法.....31
3.3 線性圖（Linear Graphs）：斜率與截距.....31
3.4 線性代數（Linear Algebra）概覽.....32
3.5 向量（Vectors）.....33
3.5.1 向量記法與幾何意義.....33
3.5.2 運動、加法與減法.....33
3.5.3 純量運算.....33
3.6 矩陣（Matrices）.....34
3.6.1 矩陣的維度.....34
3.6.2 特殊矩陣.....34

3.6.3 JavaScript 中的矩陣與 math.js.....35
3.6.4 矩陣加法、減法與純量乘除.....35
3.6.5 轉置（Transpose）.....36
3.6.6 矩陣乘法（Multiplying Matrices）.....36
3.6.7 矩陣分解.....37
3.7 張量（Tensors）.....37
3.7.1 張量的秩（Rank）.....37
3.7.2 JavaScript 中的張量運算：TensorFlow.js.....37
4. 統計與概率基礎.....39
4.1 推論統計與描述統計.....39
4.2 集中趨勢：平均數、中位數、眾數.....39
4.2.1 平均數（The Mean）.....39
4.2.2 中位數（The Median）.....40
4.2.3 眾數（The Mode）.....40
4.2.4 離群值（Outliers）.....40
4.3 離散程度：方差與標準差.....41
4.3.1 方差（Variance）.....41
4.3.2 標準差（Standard Deviation）.....42
4.4 分佈（Distribution）.....42
4.4.1 正態分佈（Normal Distribution）.....42
4.4.2 誤差幅度（Margin of Error）.....42
4.4.3 偏度（Skewness）與峰度（Kurtosis）.....43
4.5 概率（Probability）.....43
4.5.1 擲硬幣.....43
4.5.2 擲骰子.....43
4.5.3 六個球.....44
4.5.4 抽到國王牌.....44
5. 機器學習的核心概念：數據、術語、學習類型、聚類與迴歸.....45

5.1 數據（Data）.....45
5.1.1 甚麼是數據？.....45
5.1.2 儲存數據：樓價與面積數據集.....45
5.1.3 定量與定性；普查與抽樣.....46
5.1.4 大數據與數據挖掘.....46
5.2 散點圖與數據關係.....46
5.3 機器學習術語：關係、標籤、特徵、模型、訓練與推論.....47
5.3.1 關係（Relationships）.....47
5.3.2 標籤與特徵.....47
5.3.3 模型、訓練、推論與兩個階段.....47
5.4 學習類型.....48
5.5 聚類（Clustering）與相關系數.....48
5.5.1 聚類的概念.....48
5.5.2 相關系數（Correlation Coefficient, r）.....49
5.6 線性迴歸（Linear Regressions）.....49
5.6.1 如何從散點預測未來價格？.....50
5.6.2 多項式迴歸（Polynomial Regression）.....51
6. 感知器（Perceptron）：原理、程式實作、訓練與測試.....52
6.1 Frank Rosenblatt 與感知器的起源.....52
6.2 感知器的設計.....52
6.2.1 去不去音樂會？一個直觀例子.....52
6.2.2 感知器演算法（The Perceptron Algorithm）.....52
6.3 感知器的組成部分.....53
6.4 感知器如何學習；感知器的限制.....54
6.5 模式識別：用感知器把點分成「線上」與「線下」.....54
6.5.1 程式步驟總覽.....54
6.5.2 逐步程式碼.....55
6.6 訓練感知器.....56

6.6.1 建立感知器物件.....56
6.6.2 加入啟動函數.....56
6.6.3 建立訓練函數.....57
6.6.4 建立自己的程式庫並使用.....58
6.7 測試與調校感知器.....59
7. 學習即迴圈：成本函數與梯度下降（Gradient Descent）.....61
7.1 學習就是迴圈（Learning is Looping）.....61
7.2 梯度下降的概念.....61
7.3 訓練器物件（Trainer Object）.....61
7.4 成本函數（Cost Function）.....61
7.5 訓練函數與更新權重函數.....62
7.6 建立自己的程式庫.....63
7.7 比較：感知器訓練 vs 梯度下降訓練器.....64
8. 機器學習的程式語言與 JavaScript 生態.....66
8.1 機器學習常用的程式語言.....66
8.2 為何用 JavaScript 做機器學習？.....66
8.2.1 JavaScript 機器學習與數學程式庫一覽.....67
8.3 Brain.js 實作.....68
8.3.1 第一個神經網絡：學習 [0,0]→zero、[0,1]→one、[1,0]→one、[1,1]→zero.....68
8.3.2 預測顏色的深淺（對比度）.....68
8.4 機器學習應用實例.....69
8.4.1 圖像分類示範.....69
8.4.2 人工音樂智能：David Cope 與 EMI.....70
8.4.3 Project Baseline.....70
8.4.4 數據科學家.....70
8.4.5 人工健康智能與 FDA 聲明.....70
9. TensorFlow.js：張量、運算、模型與視覺化.....71
9.1 甚麼是 TensorFlow.js？.....71

9.2 張量（Tensors）.....71
9.2.1 建立張量.....72
9.2.2 讀取張量的值與屬性.....72
9.2.3 張量的數據類型.....73
9.3 張量運算（TensorFlow Operations）.....73
9.4 TensorFlow 模型（Models）.....73
9.4.1 TensorFlow 項目的典型工作流程.....73
9.4.2 範例：學習直線 Y = 1.2X + 5.....74
9.4.3 TensorFlow 優化器一覽.....75
9.5 TensorFlow.js Visor（tfjs-vis）.....76
10. TensorFlow.js 完整範例：汽車油耗與樓價預測.....78
10.1 範例一：以馬力預測每加侖英里數（汽車數據）.....78
10.1.1 數據收集.....78
10.1.2 清理數據：移除不需要的數據與錯誤.....79
10.1.3 擷取數據與繪圖.....79
10.1.4 洗牌（Shuffle）數據.....80
10.1.5 轉換成張量.....80
10.1.6 數據正規化（Normalization）.....80
10.1.7 定義模型.....80
10.1.8 編譯模型.....81
10.1.9 訓練模型.....81
10.1.10 測試模型：反正規化與繪製結果.....82
10.2 範例二：以房間數預測樓價（房屋數據）.....82
10.2.1 數據收集.....82
10.2.2 與範例一的差異.....83
10.3 兩個範例的完整流程總覽.....83
11. JavaScript 圖表與視覺化程式庫.....85
11.1 HTML Canvas 與自製繪圖器.....85

11.1.1 直接在 Canvas 上繪圖.....85
11.1.2 建立繪圖器物件（XYPlotter）.....86
11.2 Plotly.js.....87
11.3 Chart.js.....89
11.3.1 如何使用.....89
11.3.2 範例一覽.....89
11.4 Google Chart.....91
11.5 D3.js.....92
11.6 五種繪圖方式的比較.....93
12. 來源問題與不一致之處總覽.....95
13. 重點總結.....96
13.1 概念層面.....96
13.2 數學與統計工具.....96
13.3 機器學習實務.....97
13.4 工具與實作.....97
14. 資料來源清單.....98
14.1 主要來源：W3Schools AI／Machine Learning 教程（擷取日期 2026-10-14）.....98
14.2 補充來源（僅用於【補充說明】與【來源問題】方框中的核對）.....100
附錄：中英術語對照表.....100

0. 前言與使用說明

0.1 本講義的目的與範圍

本講義依據 W3Schools 的「AI／Machine Learning 教程」（網域 https://www.w3schools.com/ai/，共 55 個教學頁面）編寫，以繁體中文（香港用語）有系統地整理教程中的所有實質內容，包括：人工智能與機器學習的基本概念、歷史脈絡、數學與統計基礎、感知器（Perceptron）與神經網絡的原理及程式實作、梯度下降（Gradient Descent）的訓練迴圈、JavaScript 機器學習生態（Brain.js、ml5.js、TensorFlow.js）、兩個完整的 TensorFlow.js 範例，以及用於繪製機器學習圖表的 JavaScript 圖形程式庫。

講義並非單純摘要，而是將原教程較為零散、口語化的敘述重新組織成由淺入深的學習順序，補充必要的解釋與步驟說明，並在適當位置加入比較表與練習提示。所有重要的名稱、年份、數字、公式及程式碼均按原文保留，技術名詞首次出現時附上英文原名。

0.2 關於「附件資料」的重要說明

> **【來源問題／不一致之處】** 使用者的指示提及「附件資料（attached materials）」，但本次對話的工作區內**並沒有任何附加檔案**，系統亦沒有列出任何附件。因此，本講義**只能以指示中列出的網上來源（W3Schools AI 教程整個網域）作為主要參考資料**。如果原本打算提供其他講義、簡報或筆記，請於下一輪補上，以便將其內容整合並更新本講義。

0.3 來源引用與標示慣例

為方便查證，本講義採用以下引用方式：

- 每個 W3Schools 教學頁面均獲編配一個來源編號 **S01 至 S55**（完整清單見第 14 節），並按網站導覽列順序排列。
- 正文中的引用寫作（S08「The Perceptron Algorithm」），表示資料出自編號 S08 的頁面中名為 The Perceptron Algorithm 的小節。W3Schools 網頁沒有頁碼，故一律以**小節標題**代替頁碼。
- 網頁內容擷取日期為 **2026 年 10 月 14 日**。W3Schools 會不定期修訂內容，日後查閱時可能略有差異。

講義中以四種着色方框區分不同性質的內容：

方框標示	意義
【來源重點】	直接整理自 W3Schools 原文的關鍵要點，屬**來源支持的資訊**。
【補充說明（非來源內容）】	由講義編者加入的解釋、推導、背景或對照資料，**並非出自 W3Schools 教程**；凡引用外部資料者均另行註明出處。
【來源問題／不一致之處】	原教程中出現的錯漏、前後矛盾、含糊不清或與其他可靠資料不符的地方，供讀者留意。

方框標示	意義
【學習提示】	學習方法、常見誤解或練習建議。

沒有方框標示的正文，若附有（Sxx「小節」）引用，即屬來源支持的內容；若為承上啟下的說明文字，則屬編者整理，不會加入新的事實主張。

0.4 原教程中無法於講義重現的元素

- **互動範例（Try it Yourself）**：W3Schools 幾乎每個程式範例都附有可即時執行的互動編輯器，講義只能保留程式碼與文字說明；讀者可按第 14 節的網址親自試驗。
- **圖片與動畫**：例如神經網絡層級示意圖、向量加法圖、正態分佈曲線圖、Sophia 機械人動畫等，講義以文字描述其內容。
- **嵌入式互動元件**：例如「Click to Train Me」感知器訓練示範、線性迴歸「訓練 100／200／300／500 次」按鈕、二進制加法計算器、圖像分類器等。
- **影片連結**：例如 Royal Caribbean 遊輪機械人調酒師、Amazon 倉庫機械人、咖啡機械人等影片。

0.5 建議的閱讀順序

本講義已把原教程的八個導覽分區（Machine Learning、TensorFlow、Example 1、Example 2、JS Graphics、History、Mathematics、Statistics）重新編排為由基礎到進階的學習路線：

1. 第 1 節：先建立人工智能、機器學習、神經網絡與深度學習之間的層級關係。
2. 第 2 節：了解智能、語言、數字、計算、機械人與人工智能的歷史背景，以及心智理論與職業取代等議題。
3. 第 3、4 節：補足線性函數、線性代數（向量、矩陣、張量）以及統計與概率等數學工具。
4. 第 5 節：掌握機器學習的數據、術語（標籤、特徵、模型、訓練、推論）、學習類型、聚類、相關性與迴歸。
5. 第 6、7 節：深入感知器的運作、訓練與測試，並理解成本函數與梯度下降如何驅動「學習」。
6. 第 8 至 11 節：進入實作層面——程式語言選擇、JavaScript 程式庫、TensorFlow.js、完整範例與圖表繪製。
7. 第 12 至 14 節：查閱來源問題總覽、重點總結與資料來源清單；附錄提供中英術語對照。

> **【學習提示】** 原教程以 JavaScript 為實作語言，所有程式範例都在瀏覽器執行。閱讀第 6 節起的程式碼時，建議具備基本的 JavaScript 知識（變數、陣列、迴圈、函數與物件）。若尚未接觸過 JavaScript，可先閱讀概念部分，再回頭看程式碼。

1. 人工智能、機器學習與深度學習的全貌

本節整合 W3Schools 教程首頁（S01）、「What is AI?」（S02）與「Deep Learning」（S17）三個頁面的內容，目的是先建立一幅清晰的「全景圖」：人工智能（Artificial Intelligence, AI）是甚麼？機器學習（Machine Learning, ML）、神經網絡（Neural Networks, NN）與深度學習（Deep Learning, DL）又分別處於這幅圖的哪個位置？

1.1 甚麼是人工智能（Artificial Intelligence）？

W3Schools 指出，人工智能的含義是機器可以在以下方面**模仿人類**（S02「What is Artificial Intelligence?」）：

- 說話（Talking）
- 思考（Thinking）
- 學習（Learning）
- 計劃（Planning）
- 理解（Understanding）

人工智能亦稱為**機器智能（Machine Intelligence）**或**電腦智能（Computer Intelligence）**（S02）。教程以「人工智能是與人類智能的對照（Artificial Intelligence Is a Contrast to Human Intelligence）」作為小節標題，強調 AI 是相對於自然智能而言的概念。

教程列出多個機構或人物對 AI／ML 的定義，整理如下（S02「What is Artificial Intelligence?」）：

出處（年份）	定義（中譯）	原文要點
Arthur Samuel（1959）	「機器學習是電腦科學的一個子領域，使電腦不需被明確編程便具備學習能力。」	出自 IBM Journal of Research and Development, Vol. 3, 1959
Wikipedia（2022）	人工智能是由機器所展示的智能；有別於人類和動物所展示、涉及意識與情感的自然智能。	Intelligence demonstrated by machines
Investopedia（2022）	人工智能是指在機器中模擬人類智能，這些機器被編程為像人類一樣思考並模仿其行為。	Simulation of human intelligence in machines
IBM（2022）	人工智能利用電腦和機器，模仿人類心智的解難與決策能力。	Mimic problem-solving and decision-making capabilities
Britannica（2022）	人工智能是數碼電腦或電腦控制的機械人執行通常與智能生物相關的任務的能力，例如推理、發現意義、概括或從過往經驗中學習。	Reason, discover meaning, generalize, learn from past experience

【來源問題／不一致之處】 Arthur Samuel 這句名言在 S02 與 S17 兩頁出現，措辭略有不同：S02 寫作 “a subfield of computer science that gives computers the ability to learn without being programmed”，S17 則寫作 “a field of study that gives computers the ability to learn without being programmed”。兩者意思一致，講義按出現位置分別保留。

1.2 人工智能的層級結構

教程首頁與 S02 均以一幅同心層級圖呈現 AI 的範疇。圖中由外至內（或由廣至專）排列為（S01、S02「Artificial Intelligence (AI)」）：

1. 人工智能（Artificial Intelligence）
2. 窄人工智能（Narrow AI）
3. 機器學習（Machine Learning）
4. 神經網絡（Neural Networks）
5. 大數據（Big Data）
6. 深度學習（Deep Learning）
7. 強人工智能（Strong AI）

教程對此的文字說明是：「人工智能是一門科學學科，涵蓋多個數據科學（Data Science）領域，範圍由窄人工智能到強人工智能，包括機器學習、深度學習、大數據與數據挖掘（Data Mining）。」（S02「Artificial Intelligence (AI)」；S05「Data Scientists」亦有相同句子）

> **【補充說明（非來源內容）】** W3Schools 的層級圖是概念性的示意，並非嚴格的集合包含關係——例如「大數據」並不是「神經網絡」的子集，而是支撐深度學習的重要資源。較常見的嚴謹表述是：**深度學習 ⊂ 機器學習 ⊂ 人工智能**，而「窄 AI／強 AI」則是按能力範圍對 AI 系統的另一種分類方式。

1.3 窄人工智能（Narrow AI）與強人工智能（Strong AI）

1.3.1 窄人工智能（又稱弱人工智能 Weak AI）

窄人工智能只限於狹窄（特定）的範疇，亦即我們今天身邊大部分的 AI（S02「Narrow AI」）。教程列舉的例子有：

類別（編者歸納）	教程列舉的例子
通訊與文字	電郵垃圾郵件過濾（Email spam Filters）、文字轉語音（Text to Speech）、語音識別（Speech Recognition）、文字自動校正（Text Autocorrect）、自動翻譯（Automated Translation）、聊天機械人（Chatbots）、自然語言處理（NLP – Natural Language Processing）
視覺與感知	人臉偵測（Face Detection）、視覺感知（Visual Perception）
出行與機械	自動駕駛汽車（Self Driving Cars）、Google Maps、機械人（Robots）、飛行無人機（Flying Drones）
商業與金融	電子支付（E-Payment）、自動化投資（Automated Investment）、搜尋演算法（Search Algorithms）、社交媒體（Social Media）

類別（編者歸納）	教程列舉的例子
語音助理與推薦	IBM 的 Dr. Watson、Apple 的 Siri、Microsoft 的 Cortana、Amazon 的 Alexa、Netflix 的推薦系統（Netflix's Recommendations）

教程以兩句話對比弱 AI 與強 AI（S02「Narrow AI」）：

- **弱人工智能（Weak AI）**：為**模擬**（simulate）人類智能而建造。
- **強人工智能（Strong AI）**：為**複製**（copy）人類智能而建造。

1.3.2 強人工智能

強人工智能是能夠模仿人類智能的 AI 類型，具備思考、計劃、學習與溝通的能力。教程形容它是「理論上 AI 的下一個層次：真正的智能（True Intelligence）」，並會朝向擁有自我意識（self-awareness）、意識（consciousness）與客觀思想（objective thoughts）的機器發展（S02「Strong AI」）。

> *One need not decide if a machine can "think". One need only decide if a machine can act as intelligently as a human.*
> *（不必判定機器能否「思考」，只需判定機器能否表現得與人類同樣聰明。）*
>
> — Alan Turing（引自 S02）

> **【補充說明（非來源內容）】** 這段話概括了圖靈 1950 年論文〈Computing Machinery and Intelligence〉的核心思想，即以行為表現（後世稱為「圖靈測試」）取代「機器能否思考」的哲學爭論。該論文亦是 S42 歷史年表的第一項（1950 年）。

1.4 機器學習與傳統編程的根本分別

這是整個教程最核心的一個對比，分別在 S01「Machine Learning (ML)」與 S17「Deep Learning」兩處出現：

方式	公式	說明
傳統編程（Traditional / Classical Programming）	**數據 + 演算法 = 結果**（Data + Algorithms = Results）	人先寫好演算法（程式），電腦把數據代入演算法算出結果。
機器學習（Machine Learning）	**數據 + 結果 = 演算法**（Data + Results = Algorithms）	人提供數據與對應的正確結果，由電腦自行「學出」演算法（模型）。

因此，機器學習是「從數據與結果中創造演算法」，而不是「用演算法處理數據」（S01）。教程的副標題是：「Learning machines to imitate human intelligence」（讓機器學習以模仿人類智能）。

教程特別澄清一個常見誤解：**機器學習常被視為等同於人工智能，這是不正確的**。機器學習是人工智能的子集，是 AI 中「以數據教導機器」的一門學科（S17「Machine Learning」）。

1.5 神經網絡（Neural Networks）的基本概念

根據教程首頁，神經網絡是（S01「Neural Networks (NN)」）：

- 一種編程技術（A programming technique）
- 機器學習中使用的一種方法（A method used in machine learning）
- 一種會從錯誤中學習的軟件（A software that learns from mistakes）

神經網絡的設計靈感來自人腦的運作方式：神經元（neurons）彼此傳遞訊息；當神經元反覆嘗試解決某個問題時，會**強化導致成功的連接，並削弱導致失敗的連接**（S01）。

S17「Neurons」補充了生物學背景：科學家普遍認同人腦有 **800 億至 1,000 億個神經元**，它們之間有數千億個連接。神經元（又稱神經細胞 nerve cells）是大腦與神經系統的基本單位，負責接收外界輸入、向肌肉發出指令，以及在兩者之間轉換電訊號。（教程配圖來源註明為 University of Basel, Biozentrum。）

S17「Neural Networks」進一步指出：人工神經網絡（Artificial Neural Networks）通常簡稱神經網絡（NN）；神經網絡實際上是**多層感知器（multi-layer Perceptrons）**；神經網絡是深度學習的精髓，被形容為「歷史上最重要的發現之一」，能解決**演算法無法解決**的問題，例如：

- 醫療診斷（Medical Diagnosis）
- 人臉偵測（Face Detection）
- 語音識別（Voice Recognition）

1.6 從感知器到深度神經網絡：層的概念

1.6.1 感知器（Perceptron）

感知器是「踏入神經網絡的第一步」，代表**一個神經元**，只有一個輸入層而沒有隱藏層（S01「Perceptrons」）。感知器的詳細原理與程式實作見第 6 節。

1.6.2 神經網絡＝多層感知器

最簡單的神經網絡由三層組成（S01「Neural Networks」，教程以顏色區分）：

層	顏色（教程圖示）	角色
輸入層（Input layer）	黃色	黃色感知器根據輸入作出簡單決定，每個決定都送到下一層的感知器。
隱藏層（Hidden layer）	藍色	藍色感知器把第一層的結果加權後作出決定；這一層的決定比第一層更複雜、更抽象。
輸出層（Output layer）	紅色	產生最終輸出。

在「神經網絡模型」中，輸入數據（黃）先經過一個隱藏層（藍）處理，然後產生最終輸出（紅）（S01）。

1.6.3 深度神經網絡（Deep Neural Networks）

深度神經網絡由**多個隱藏層**組成，可對海量數據執行複雜運算；**每一層都以前一層的輸出作為輸入**（S01「Deep Neural Networks」）。教程舉例：光學閱讀（optical reading）以較低層辨認邊緣（edges），以較高層辨認字母（letters）。

在「深度神經網絡模型」中，輸入數據（黃）先經隱藏層（藍）處理，再經更多隱藏層（綠）修改，最後產生輸出（紅）。第三層（綠色感知器）作出「更加複雜的決定」（S01；S17「The Neural Network Model」亦有相同描述）。

> **【學習提示】** 「每一層都以前一層為輸入」與「低層辨認邊緣、高層辨認字母」是理解深度學習「層次化特徵學習」的關鍵直覺：越深的層所處理的概念越抽象。這也解釋了為何深度網絡需要更大量數據與更強運算能力（見 1.7 節的四個因素）。

1.7 深度學習（Deep Learning）

深度學習是機器學習的子集，被認為是「近年 AI 熱潮的推手」，是一種能處理圖像識別等複雜任務的進階 ML（S01「Deep Learning (DL)」）。教程以下表比較兩者：

機器學習（Machine Learning）	深度學習（Deep Learning）
AI 的子集	機器學習的子集
使用較小的數據集	使用較大的數據集
由人類訓練	自行學習
建立簡單的演算法	建立複雜的演算法

S17 指出深度學習革命大約始於 **2010 年**，此後解決了許多過去「無法解決」的問題。這場革命並非源自單一發現，而是在幾個必要條件同時成熟時「或多或少自然發生」（S17 開首）：

- 電腦速度夠快（Computers were fast enough）
- 電腦儲存容量夠大（Computer storage was big enough）
- 發明了更好的訓練方法（Better training methods were invented）
- 發明了更好的調校方法（Better tuning methods were invented）

1.8 「學習」的正式定義：Tom Mitchell 的 E–T–P 框架

教程介紹 Tom Michael Mitchell（1951 年生）：美國電腦科學家、卡內基美隆大學（Carnegie Mellon University, CMU）University Professor，曾任該校機器學習系系主任（S17「Tom Mitchell」）。他對「學習」的定義是：

> *A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E.*

> *（若一個電腦程式在某類任務 T 上，以表現量度 P 衡量的表現，隨經驗 E 而改善，則稱該程式從經驗 E 中學習。）*
>
> — Tom Mitchell（S17 註明為 1999 年）

符號	意義	教程舉例
E（Experience）	經驗	嘗試的次數（the number of times）
T（Task）	任務	駕駛汽車（driving a car）
P（Performance）	表現	好或壞（good or bad）

【來源問題／不一致之處】 S17 把這個定義標註為 “Tom Mitchell (1999)”。然而，這個廣被引用的定義出自 Mitchell 的教科書《Machine Learning》（McGraw-Hill，**1997 年**）。見 Wikipedia「Machine learning」條目及多個教材來源（詳見第 14 節補充來源）。讀者引用時宜註明 1997 年。

1.9 兩個說明「機器智能」的故事

1.9.1 Giraffe 國際象棋程式（2015）

2015 年，倫敦帝國學院（Imperial College London）學生 Matthew Lai 建立了名為 Giraffe 的神經網絡。Giraffe 只需 **72 小時**訓練，便能以**國際大師（international master）**的水平下國際象棋。教程強調：電腦下棋並不新鮮，新鮮的是**建立方式**——傳統的高水平棋藝程式需要多年開發，而 Giraffe 以神經網絡在 72 小時內建成（S17「The Giraffe Story」）。

1.9.2 「智能決策公式」

教程以一個六步驟的迴圈說明電腦如何作出「智能」決定（S17「Intelligent Decision Formula」）：

1. 儲存所有行動的結果
2. 模擬所有可能的結果
3. 把新行動與舊行動比較
4. 檢查新行動是好是壞
5. 如果新行動「沒那麼差」，就選擇它
6. 從頭再做一遍

教程的結論是：電腦能把這個過程重複執行數百萬次，這證明了電腦能作出非常聰明的決定（S17）。

【補充說明（非來源內容）】 這個「公式」其實是搜尋與試錯式學習的直觀版本，與第 7 節的梯度下降（每次迭代都朝「成本更低」的方向微調）以及 S13 提到的強化學習（根據回饋改進決策）在精神上一致。

2. 智能、計算與人工智能的歷史脈絡

W3Schools 教程的「History」分區由八個頁面組成（S37–S44），從人類認知智能的起源講到語言、數字、計算機、機械人與 AI 的發展，再延伸至職業取代與心智理論。這些內容為理解「人工智能為何是關於數字與計算」提供了背景。

2.1 人類智能的起源

約 **70,000 年前**，人類大腦發生了某種變化，開始發展「認知智能（Cognitive Intelligence）」，包括（S37 開首）：

- 能夠理解語言
- 能夠理解數字
- 能夠理解抽象思維

2.1.1 詞語、數字與語言

教程以簡單例子說明詞語與數字為何是智能發展的大躍進（S37「Words and Numbers」「Languages」）：

- 「大象」比「大動物」更具資訊量。
- 「5」或「50」比「少」或「多」更具資訊量。
- 「大橡樹後面有一隻獅子」比大叫「危險！」更具資訊量。

擁有語言很可能是人類有別於動物的關鍵特徵（S37）。

2.1.2 抽象思維（Abstract Thinking）

抽象思維是指思考不具體的事物，例如自由、意念或概念（S37「Abstract Thinking」）。教程以自由神像為例作出對比：

抽象（Abstract）	**具體（Concrete）**
「自由」的象徵（A freedom figure）	一座具體的雕像（A concrete figure）
自由、過去、未來、想像、解釋	我的嘴巴、我的指尖、一個蘋果、兩隻雞、三匹馬

許多日常行為都用到抽象思維：使用概念（Concepts）、理論（Theories）、隱喻（Metaphors）、類比（Analogies）與想像（Imagination）（S37）。

2.1.3 甚麼是智能？

教程坦言「智能沒有經證實的定義」，但可以定義為：能理解抽象思維、能理解語言、能理解數字；也可以定義為「知識的集合」（S37「What is Intelligence?」）。教程引用：

> *Intelligence could be defined as the process of obtaining knowledge by efficient means.*
> *（智能可以定義為以高效方式獲取知識的過程。）*

— Yampolsky（2013），引自 S37

AI 的挑戰在於理解甚麼是智能以及智能如何運作。教程提出：智能可能是我們大腦或基因中的某種演算法（S37）。

> **【來源問題／不一致之處】** S37 把引文作者拼作 “Yampolsky”。研究人工智能安全的學者 Roman V. Yampolskiy 的姓氏通常拼作 **Yampolskiy**；教程未提供文獻出處，講義無法核實原始出處，故按原文保留。

2.1.4 人腦是否更優越？

教程以一連串反問引導思考（S37「Is the Human Brain Superior?」）：人類的腦比動物大；大狗的腦比小狗大，大狗就更聰明嗎？鳥的腦很小，卻知道不應在秋天育雛、知道何時南飛；蜜蜂的腦更小，卻懂得溝通花蜜的位置；植物沒有腦，卻懂得繁殖和準備過冬。

2.2 語言的歷史

語言是一種**有結構的溝通系統**，是涉及使用詞語的溝通類型（S38 開首）。猿類與鯨類、鳥類與蜜蜂都會彼此溝通，但**只有人類發展出真正的語言**——沒有其他物種能用由一組詞語（含動詞與名詞）構成的句子表達意念；更驚人的是，連兒童也能掌握這項技能（S38「What is a Language?」）。

2.2.1 口語的起源

口語有多古老並不確定，因為缺乏證據而難以研究。教程提供一條線索：非洲大猿（Pan 屬與 Gorilla 屬）是我們最近的現存親屬；牠們被稱為 apes，是因為牠們會 ape（模仿）——用動作把訊息傳達出去。語言的演化應是漫長的過程，我們的祖先可能在**一百萬年前**已開始說話，但詞語較少、更多靠模仿動作，且沒有文法（S38「Spoken Languages」）。（教程配圖為老彼得・布勒哲爾的《巴別塔》。）

2.2.2 認知發展的六個方面

根據 Hauser、Chomsky 與 Fitch（2002），語言發展有六個方面（S38「Cognitive Development」）：

1. 心智理論（Theory of Mind）
2. 理解聲音訊號
3. 理解模仿
4. 理解數字
5. 理解有意圖的溝通
6. 理解非語言的表徵

2.2.3 人類語言與電腦語言

人類語言以有限的詞語組成句子；電腦語言以有限的詞語組成電腦陳述句（statements）（S38「Human Languages」「Computer Languages」）。教程的例子：

人類語言（英、意、西、德）

```
I'm going on holiday in my new car.
Vado in vacanza con la mia macchina nuova.
Me voy de vacaciones en mi auto nuevo.
Ich fahre mit meinem neuen Auto in den Urlaub.
```

電腦語言（JavaScript）

```
var points = [40, 100, 1, 5, 25, 10];
points.sort(function(a, b){return a - b});
```

2.2.4 書面語言年表

埃及文與蘇美爾文是已知最早的書面語言；而**仍在使用中的最古老書面語言是中文**（S38「Written Languages」）。教程的年表如下：

年份	語言	年份	語言
公元前 3500	蘇美爾文（Sumerian）	公元前 700	拉丁文（Latin）
公元前 3000	埃及文（Egyptian）	公元 700	古典梵文（Classical Sanskrit）
公元前 1500	中文（Chinese）	公元 500	阿拉伯文（Arabic）
公元前 1500	吠陀梵文（Vedic Sanskrit）	公元 400	哥德文（Gothic，德語系）
公元前 1500	希臘文（Greek）	公元 700	英文（English）
公元前 1000	希伯來文（Hebrew）	公元 700	日文（Japanese）
公元前 900	亞拉姆文（Aramaic）	公元 800	法文（French）
公元前 700	伊特魯里亞文（Etruscan）	公元 900	意大利文（Italian）
公元前 500	泰米爾文（Tamil）	公元 1000	西班牙文（Spanish）

> **【來源問題／不一致之處】** 原表的次序並非嚴格按年份排列（例如「公元前 500 泰米爾文」排在「公元前 700 拉丁文」之前，「公元 700 古典梵文」排在「公元 500 阿拉伯文」之前）。此外原文以 “AC” 表示公元後（一般寫法為 AD 或 CE）。講義保留原表次序與數值，僅將 AC 譯為「公元」。

2.3 數字的歷史

教程強調：要理解 AI，必須理解**數字（Numbers）與計數（Counting）**的概念——「人工智能完全是關於數字的」（S39 開首及「AI is About Numbers」）。

2.3.1 AI 關乎數字與計數

數字容易理解（1, 2, 3 … 11, 12, 13 …）；對動物的研究顯示動物也能理解一些數字（例如 2 個妻子、8 個兒子、5 隻蛋）。現代世界對數字的需求是絕對的：100 美元、Pi = 3.14、365 天、25 年、20% 稅、100 英里（S39）。

數字的概念引伸出計數的概念。想像史前的思考：如何數蘋果？如何稱量穀物？如何付款？海洋有多遠？教程說：「人工智能是人類對計算需求的結果」；計數也容易理解（2 + 2 = 4），但動物只能理解非常簡單的計數；智人把複雜計算交給電腦——「是的！電腦可以比人類聰明。」（S39「AI is About Counting」）

2.3.2 巴比倫數字：六十進制（Base 60）

教程相信巴比倫人開啟了複雜計數的發展。巴比倫數字系統有 **60 個不同的數字符號**，是**六十進制（sexagesimal, base 60）**系統（S39「Babylonian Numbers (Base 60)」）。

教程以一段虛構的「約 6,000 年前兩位巴比倫科學家的對話」說明為何需要一個數字系統——若對方不知道「7」是甚麼，便無法告訴他「我有 7 個兒子」；需要多少個數字？15 個不夠（有人兒子多於 15 個），30 個也不夠（30 歲以上的人要能說出年齡），最後定為 60（S39「Two Babylonian Scientists」）。

六十進制的特點（S39「Sexagesimal (base 60)」）：

- 1 分鐘有 60 秒；1 小時有 60 分鐘。
- 60 非常「多才多藝」：可被 1, 2, 3, 4, 5, 6, 10, 12, 15, 20, 30, 60 整除（教程兩處列法略有不同，其中一處漏了 20）。
- 巴比倫系統是**位值制（place-value system）**：左邊的數字代表較大的單位，與我們的十進制相似。例如「1,5」代表 65（1 × 60 + 5）；「3,30」代表 210（3 × 60 + 30）。
- 缺點是使用者須記住 60 個不同符號；但這個系統很聰明，很多部分沿用至今。

甚麼	我們的用法	為何
一個完整的圓	360°	6 × 60 = 360
半個圓	180°	3 × 60 = 180
一小時	60 分鐘	1 × 60 = 60 分鐘

> **【來源問題／不一致之處】** 原表第三行寫作 “One Hour | 60° | 1 x 60 = 60 minutes”，把「一小時」與「60°」並列，應為排版筆誤（一小時是 60 分鐘，而非 60 度）。講義已按語意改為「60 分鐘」。

教程進一步解釋巴比倫人為何「數到 360」，並指出這套由更早的美索不達米亞文化承傳下來的系統，留下許多至今仍用的數學與天文慣例（S39）：

- **把圓分成 360 度**：巴比倫人觀察到一年約 360 天（基於早期太陽曆），並把天體運行與圓周運動聯繫起來。

- **使用六十進制**：60 是高合成數（highly composite number），因數很多，便於處理分數與量度；360 是 60 的倍數（60 × 6）。
- **天文與曆法意義**：追蹤月亮與太陽週期（與約 360 天的一年相關）；把天空分成 12 個黃道星座，每個橫跨 30 度。
- **對時間量度的影響**：這套系統經希臘與希臘化時期天文學流傳下來，最終塑造了現代數學、幾何與計時。

六十進制的起源已失落於歷史中，但看來自古以來一直與十二進制並行使用（S39）。

2.3.3 十二進制（Dozenal, Base 12）

- 一打有 12 個；白天 12 小時、夜晚 12 小時；一年 12 個月。
- 12 可被 1, 2, 3, 4, 6, 12 整除。

如何用雙手數到 60：每根手指有 3 個指節，用拇指點數左手其餘四指的 12 個指節，便可數到 12；右手則記錄「數了幾個左手」。1 隻滿手 = 12，2 隻 = 24，3 隻 = 36，4 隻 = 48，5 隻 = 60（S39「How to Count Dozenal」）。

2.3.4 羅馬數字：十進制（Base 10）

羅馬數字起源於羅馬，在歐洲一直使用至中世紀（S39「Roman Numbers (Base 10)」）：

符號	I	V	X	L	C	D	M
數值	1	5	10	50	100	500	1000

羅馬人使用十進制，可能因為他們以 10 根手指計數。今天世界大部分地方都使用十進制：西方數字、中國數字、婆羅米數字、希臘數字、希伯來數字……位值的意義：5 代表 5；50 代表 5 個 10；500 代表 5 個 100；57 代表 5 個 10 加 7；75 代表 7 個 10 加 5（S39）。

2.3.5 數碼數字：二進制（Base 2）

人類能用十進制計數，電腦卻不能。電力不會計數，只有「開」與「關」；電腦用電或磁來表示開／關，因此使用像 11100110 這樣的**數碼數字（Digital Numbers）**，而非像 230 這樣的十進制數字。數碼數字亦稱**二進制數字（Binary Numbers）**（S39「Digital Numbers (Base 2)」）。教程最後指出：電腦能處理無限量的數字，人類不能（S39「Man vs. Computer」）。

> **【補充說明（非來源內容）】** 11100110（二進制）= 128 + 64 + 32 + 4 + 2 = **230**（十進制）。教程兩處（S39、S40）都用這對數字作例子，正好互相對應。

2.4 計算的歷史

S40 以「算盤 → 類比計算機 → 數碼計算機 → 電子計算機 → 電腦速度」為主線。

2.4.1 算盤（Abacus）

- **巴比倫算盤**的發明是為了**縮短計算時間**。公元前 2700–2300 年間可能出現了第一個算盤——一個由連續欄位組成的桌面，各欄代表六十進制數字系統的位數（S40「The First Abacus」）。
- **羅馬算盤（Abacus 2.0）**使用十進制羅馬數字，是早期巴比倫算盤的便攜式、十進制版本，被教程形容為「世界上第一部手持電腦」，供羅馬工程師、商人與稅吏使用（S40「Abacus 2.0」；配圖出自 1911 年《大英百科全書》，公有領域）。

2.4.2 類比計算機（Analog Computers）

- **差分機（Difference Engine，Charles Babbage，1822 年）**：機械式機器，用以縮短計算複雜數學函數的時間。
- **分析機（Analytical Engine，Charles Babbage，1833 年）**：機械式機器，設計上已具備算術、邏輯與記憶等現代電腦元素。
- 兩者都用十進制的機械齒輪進行計算（S40「Analog Computers」；配圖為倫敦科學博物館的分析機）。

2.4.3 數碼計算機（Digital Computers）

數碼計算機使用 0／1 開關進行計算，以二進制值（如 11100110）運算，對比類比值（如 230）。頁面附有一個互動的二進制加法示範（S40「Digital Computers」）。

第一部電動數碼計算機由 Konrad Zuse 於 **1941 年**在德國設計並建造，使用 **2,600 個電磁繼電器（electrical relays）**作為 0／1 開關，時鐘速度約 **5 Hz**（S40；配圖為慕尼黑德意志博物館的 Z3 複製品）。

2.4.4 電子計算機的三個世代

世代	年代	開關元件	速度特點
第一代	1945–1950	真空管（vacuum tubes）	比繼電器快得多；時鐘速度 500 kHz 至 1 MHz
第二代	1950–1960	晶體管（transistors）	比真空管快得多（配圖：MIT TX-0 的晶體管）
第三代	1960 起	集成電路（integrated circuits）	比晶體管快得多

2.4.5 電腦速度

教程概括（S40「Computer Speed」）：第一部電動電腦每秒執行 5 條指令；第一部電子電腦每秒 5,000 條；第一部個人電腦每秒 500 萬條；AMD 是第一部達到每秒 10 億條指令的 PC；今天 iPhone 12 每秒可執行 110 億條指令。

年份	電腦	每秒指令數	每條指令位元數
1941	Z3	5	4
1945	ENIAC	5,000	8
1981	IBM PC	5,000,000	16
1995	Intel Pentium PC	100,000,000	32
2000	AMD PC	1,000,000,000	64
2020	（未註明）	10,000,000,000	128

【來源問題／不一致之處】 原表以歐陸格式「5.000.000」書寫千分位，講義改為香港慣用的「5,000,000」。另外，正文說 iPhone 12 每秒 110 億條指令，表中 2020 年一行則寫 100 億，兩者略有出入；表中 2020 年一行亦沒有填寫電腦名稱。

教程從歷史中歸納（S40「When Did it Start? Where Will it End?」）：電腦是為了計算而設計；Charles Babbage 製造了第一部機械電腦；最早的數碼電腦使用機電開關；真空管比機電開關快得多；晶體管比真空管快得多；集成電路比晶體管快得多；明天的電腦會比人腦更快？

> *Computers are useless because they can only give you answers.（電腦毫無用處，因為它們只會給你答案。）*
>
> — Pablo Picasso（引自 S40）

比人類更聰明？若相信摩爾定律（Moore's Law）：2020 年電腦具備老鼠大腦的容量；2030 年電腦可能具備人類大腦的容量（S40「Smarter Than Humans?」；來源註明為 Ray Kurzweil）。

2.5 機械人的歷史

2.5.1 工業機械人

- 工業機械人已存在超過 50 年。第一項機械人專利於 **1954 年**申請、**1961 年**獲批（S41「Industrial Robots」）。
- **1969 年** Victor Scheinman 在史丹福大學發明 **Stanford Arm**；**1972 年**為 MIT 人工智能實驗室設計 **MIT Arm**。
- 這些「機械人」不被視為具有智能，只是容許 **6 軸**手臂運動的電動機器；但新設計讓機器能跟隨預設路徑，開啟了噴漆、焊接與裝配等「機械人工作」。
- 先驅機械人公司 **Unimation**（在通用汽車 General Motors 支持下）把 Scheinman 的設計推出市場，名為 **PUMA（Programmable Universal Machine for Assembly）**。
- 大部分工業機械人都是非智能的；現代機械人之所以稱為自主或半自主，是因為編程後不需太多人工輸入。機械人可以被編程做許多不同事情（如裝配線），但**永遠不會改變自己正在做的事**，會一直做同一工作直至被關掉。

【來源重點】 S41 明確指出：**機械人與人工智能是兩回事**；機械人技術不是人工智能的子集。機械人是

> 實體的東西——經過 50 年發展，幾乎任何東西（收音機、手錶、電話，甚至機械人）都可以編程。

2.5.2 AI 機械人

- 人工智能可以內置於機械人中，是未來機械人學一個非常令人興奮的領域。
- Hanson Robotics 的 **Sophia** 體現了人們對 AI 未來的一些夢想：她結合了科學、工程與藝術，是「一個由人手打造、描繪 AI 與機械人學未來的科幻角色」（S41「AI Robots」）。
- **AI 機械人能否社交互動？** 可以。MIT 的機械人 **Kismet** 被編程為理解身體語言與語調；其創造者研究人類與嬰兒如何基於語氣和視覺線索互動。Kismet 可能是類人學習系統的基礎（S41「Can AI Robots Interact Socially?」）。

2.6 人工智能大事年表

S42「History of AI and ML」提供了以下年表：

年份	事件
1950	Alan Turing 發表〈Computing Machinery and Intelligence〉
1952	Arthur Samuel 開發能自我學習的西洋跳棋（checkers）程式
1956	John McCarthy 在一個會議上使用「Artificial Intelligence」一詞
1957	第一種用於數值與科學計算的程式語言（FORTRAN）
1958	第一種 AI 程式語言（LISP）
1959	Arthur Samuel 使用「Machine Learning」一詞
1959	John McCarthy 與 Marvin Minsky 創立 MIT Artificial Intelligence Project
1961	第一部工業機械人（Unimate）在通用汽車的裝配線上投入使用
1965	Joseph Weizenbaum 的 ELIZA——第一個能就任何話題交談的程式
1972	第一種邏輯程式語言（PROLOG）
1991	美軍在波斯灣戰爭中使用 DART（自動化後勤規劃與排程）
1997	IBM 的 Deep Blue 擊敗國際象棋世界冠軍
2002	第一部掃地機械人（Roomba）
2005	自動駕駛汽車 STANLEY 贏得 DARPA 挑戰賽
2008	語音識別取得突破（Google）
2011	神經網絡在交通標誌識別上勝過人類（99.46% 對 99.22%）
2011	Apple Siri
2011	IBM 的 Watson 贏得問答節目 Jeopardy!
2014	Amazon Alexa
2014	Microsoft Cortana

年份	事件
2014	Google 自動駕駛汽車通過州駕駛考試
2015	Google AlphaGo 在圍棋上擊敗多位人類冠軍
2016	Hanson Robotics 的人形機械人 Sofia（Sophia）

【來源問題／不一致之處】 年表最後一項把 Sophia 拼作 “Sofia”，而 S41 及圖片檔名則用 “Sophia”。講義統一使用 Sophia。

2.6.1 為何是現在？（Why AI Now?）

教程稱 John McCarthy 為「人工智能之父」，他在 1950 年代中期創造「Artificial Intelligence」一詞，並將其定義為「製造智能機器的科學（the science of making intelligent machines）」。演算法自那時起已經存在，為何 AI 現在才更受關注？教程的答案是過去欠缺四個條件（S42「Why AI Now?」）：

- 運算能力不夠強
- 電腦儲存容量不夠大
- 沒有大數據可用
- 沒有高速互聯網

另一股強大的推動力是大企業（Google、Microsoft、Facebook、YouTube）的大量投資——因為它們的數據集已大到無法用傳統方式處理（S42）。

【學習提示】 這四個條件與 S17 所列「深度學習革命的四個因素」（電腦夠快、儲存夠大、更好的訓練方法、更好的調校方法）互相呼應，可一併記憶。

2.6.2 人與機器的對比及開放問題

人（Man）	電腦（Computer）
聰明（Smart）	愚笨（Stupid）
慢（Slow）	快（Fast）
不準確（Inaccurate）	準確（Accurate）

研究 AI 會引出許多有趣的問題：「電腦能像人一樣思考嗎？」「電腦能比人更聰明嗎？」「電腦能接管世界嗎？」機器已能理解口頭指令、辨認人臉、駕駛汽車，玩遊戲也比我們強——它們還要多久才會「走在我們中間」？（S42「Interesting Questions」）

2.6.3 人工智能電影

S42 以一個表格列出與 AI 相關的電影與劇集，部分附有簡介。講義整理如下：

年份	作品	教程附註（撮要）
1927	Metropolis（大都會）	德國科幻劇情片；被譽為史上最偉大、最具影響力的電影之一；2001 年成為首部列入 UNESCO 世界記憶名錄的電影。
1951	The Day the Earth Stood Still	美國科幻片；1995 年獲美國國家電影登記處評為具文化、歷史或美學意義。
1968	2001: A Space Odyssey	史詩科幻片；1991 年獲美國國會圖書館選入國家電影登記處。
1973	Westworld	美國科幻西部片；成人主題樂園內有三個由與真人無異的人形機械人構成的世界（西部、中世紀、羅馬）。
1977	Star Wars	史詩太空歌劇；第 50 屆奧斯卡贏得 7 項獎項；1980、1983 年續集完成三部曲；1989 年選入國家電影登記處。
1977	Demon Seed	—
1982	Blade Runner	—
1983	WarGames	—
1984	The Terminator	—
1985	D.A.R.Y.L.	DARYL（Data-Analyzing Robot Youth Lifeform）是政府創造的人工智能實驗。
1986	Short Circuit	—
1987	RoboCop	—
1994	Star Trek	—
1999	Bicentennial Man	—
1999	The Matrix	科幻動作片；2012 年選入美國國家電影登記處。
2001	A.I.（Spielberg）	—
2004	I, Robot	—
2009	Moon	—
2013	Her	科幻愛情劇情片；Theodore 與以女聲呈現的人工智能虛擬助理 Samantha 發展關係，讓人一窺未來的人工助理，以及人甚至可能愛上它們。
2013	The Machine	—
2014	Autómata	—
2014	The Imitation Game	歷史劇情片；改編自 Andrew Hodges 的傳記《Alan Turing: The Enigma》；片名引自圖靈 1950 年論文中提出、用以回答「機器能否思考」的「模仿遊戲」。
2015	Ex Machina	—
2016	Westworld（HBO 劇集）	主題樂園由被編程為滿足客人一切慾望的「接待員」（與真人無異的生物機械人）構成。
2016	Infinity Chamber	科幻片；一名以電腦病毒破壞政府行動的男子被關在由 AI 電腦 Howard 監管的自動化拘留設施內。

年份	作品	教程附註（撮要）
2019	I Am Mother	—
2019	iHuman	關於 AI、社會控制與權力的紀錄片：這項科技如何改變我們的生活、社會與未來。
2020	Coded Bias	詳盡的紀錄片，探討 AI 以及嵌入這項科技中的偏見。

【來源問題／不一致之處】 原表中 1985 年的作品寫作 “D.A.R.Y.”，按其附註中的 DARYL 應為電影《D.A.R.Y.L.》；講義已補正。

2.7 心智理論（Theory of Mind）與 AI 的未來階段

S44 以「思想、感受、情緒、自我意識、同理心」五個關鍵詞開首，探討「心智」與 AI 的距離。

2.7.1 甚麼是心智？認知科學與心智模型

- 心智（Mind）沒有經證實的定義。可定義為「智能的一個實例（instantiation of intelligence）」，或「知識的集合」。教程追問：思考、感受與意義是心智還是技能（know-how）？心智只是一部大電腦嗎？（S44「What is Mind?」）
- **認知科學（Cognitive Science）**研究心智過程；認知科學家研究智能與行為，聚焦於腦細胞如何處理和轉換資訊，並嘗試學習如何開發智能電腦演算法（S44「Cognitive Science」）。
- **心智模型（Mental Model）**是外在現實的內在圖像。科學家預期模型在推理與決策（認知）中扮演重要角色。Kenneth Craik 於 **1943 年**提出，心智在嘗試預測事件時會建構現實的「小型模型（small-scale models）」（S44「The Mental Model」）。

> *The image of the world around us, which we carry in our head, is just a model. Nobody in his head imagines all the world, government or country. He has only selected concepts, and relationships between them, and uses those to represent the real system.*
> *（我們腦中所載的周遭世界圖像只是一個模型。沒有人會在腦中想像整個世界、政府或國家；他只是選取了若干概念及其關係，用以代表真實系統。）*
>
> — Jay Wright Forrester，1971（引自 S44）

2.7.2 AI 能否成為人？四個階段

科學家正嘗試找出人類智能與人工智能的分野。教程列出一條時間線（S44「Can AI Be Human?」）：

年份	階段	說明（S44 各小節）
2000	反應式機器（Reactive Machines）	早期 AI 系統是反應式的，**不能利用過往經驗**。1997 年反應式機器 IBM Deep Blue 擊敗國際象棋世界冠軍——它不會思考，但儲存了棋盤資訊與棋子走法規則，並被編程為計算每一步以求勝。

年份	階段	說明（S44 各小節）
2015	機器學習（Machine Learning）	今天的 AI 系統能運用一些過往資訊。例如自動駕駛汽車把預先編程的資訊與學習駕駛時收集的資訊結合。
2030	心智理論（Theory of Mind）	心理學術語，指個體的同理心與理解他人的能力——意識到他人與自己一樣，各有需要與意圖。語言使用者能溝通不具體的事物（需要、意念、概念）。
2050	自我意識（Self-Awareness）	未來 AI 系統必須學會理解每個人（人與 AI 物件）都有思想與感受，並懂得調整行為以「走在我們中間」。AI 成為人之前的最後一步是**機器意識（machine consciousness）**；在我們對人腦、記憶與智能有更多了解之前，無法建構這樣的軟件。

劍橋大學心理學教授 Simon Baron-Cohen（1999）根據早期人類活動的知識，主張「心智理論」必定**先於語言**出現；這些活動包括（S44「Theory of Mind」）：

- 教導
- 建立共同目標
- 建立共同計劃
- 有意圖的溝通
- 有意圖地分享話題
- 有意圖地分享焦點
- 有意圖的說服
- 有意圖的假裝
- 有意圖的欺騙

> **【補充說明（非來源內容）】** S44 的「2000／2015／2030／2050」只是教程提出的概括性展望，並非任何機構的正式預測；2030 與 2050 兩項屬推測性質。

2.8 人工智能與職業取代

2013 年，牛津學者 Carl Benedikt Frey 與 Michael A. Osborne 發表研究，估計 **47%** 的職業將在**二十年內**被電腦取代（S43 開首；來源註明為牛津的 The_Future_of_Employment.pdf）。教程按職業類別摘錄其「電腦化概率」：

類別	職業（被電腦化的概率）	教程評語
簿記（Bookkeeping）	報稅員 99%；帳務文員 99%；簿記文員 98%；信貸分析員 98%	「他們說對了」——今天簿記已自動化；對這些工作而言，電腦比人便宜得多。
銷售與客戶	電話推銷員 99%；收銀員 97%；上門推銷員 94%；保險銷售代理 92%；銷售代表 85%；財務顧問 58%；客戶服務 55%；銷售代理 54%	網購正在接管；電話銷售的轉換率並不吸引。

類別	職業（被電腦化的概率）	教程評語
生產工人	包裝機操作員 98%；印刷裝訂及加工工人 95%；工業貨車操作員 93%；生產工人 92%；食物準備工人 87%	裝配線機械人已存在 50 年；今天任何人都能為智能機械人編程；Amazon 正以機械人取代職位。
讀寫類	數據輸入員 99%；校對員 84%；圖書館員 65%；電影及影片剪接師 54%	校對軟件無處不在，從拼寫檢查到 Hemingway App。
餐飲	快餐廚師 81%；調酒師 77%；洗碗工 77%	咖啡機械人能取代許多咖啡師（教程提及「5 Best Robot Coffee Baristas 2020」及 Royal Caribbean 遊輪的機械人調酒師）。
郵政服務	郵政服務文員 95%；郵件分揀員 79%；郵政局長 75%；郵差 68%	—
飛機機師	機師 55%	軍方今天已使用無人機；明天 AI 將取代貨機機師；客機將只有一名機師，第二名機師會是自動駕駛。
司機／運輸	的士司機 69%；貨車司機 69%；巴士司機 67%	自動駕駛汽車已是現實；潛在的職位流失驚人；AI 很快可能取代數以百萬計的運輸職位。

教程另引述 **Oxford Economics（2019）**的研究：機械人到 2030 年可能取代 **2,000 萬個**職位；到 2030 年中國可能有 **1,400 萬部**機械人在工作（S43「Robots could take 20 million jobs by 2030」）。

> **【補充說明（非來源內容）】** Frey 與 Osborne 的原文（2013 年 9 月 17 日工作論文，牛津馬丁學院）是以高斯過程分類器估計 **702 種職業**的電腦化概率，結論是「約 47% 的**美國總就業人數**處於高風險」，而非 S43 所說「47% 的所有職業」；「二十年內」亦屬媒體對「高風險」類別的常見詮釋。此外，S43 中「他們說對了」等評語屬 W3Schools 的觀點，而非研究結論。（外部來源：Oxford Martin School 出版頁面，見第 14 節。）

3. 機器學習的數學基礎：線性函數與線性代數

教程的「Mathematics」分區（S45–S50）聲稱「機器學習＝數學」：每一個 ML 成功背後都有數學，所有 ML 模型都以數學的解法與意念建構，而 ML 的目的是「建立模型以理解思考」（S45「Machine Learning = Mathematics」）。本節整理線性函數、線性圖、線性代數（純量、向量、矩陣、張量）；統計與概率則見第 4 節。

3.1 機器學習涉及的數學分支與相關職業

教程列出 ML 涉及的主要數學分支（S45 開首）：線性函數（Linear Functions）、線性圖形（Linear Graphics）、線性代數（Linear Algebra）、概率（Probability）、統計（Statistics）。

若想投身以下 ML 職業，教程建議專注於這些數學概念（S45）：

- 數據科學家（Data Scientist）
- 機器學習工程師（Machine Learning Engineer）
- 機械人科學家（Robot Scientist）
- 數據分析師（Data Analyst）
- 自然語言專家（Natural Language Expert）
- 深度學習科學家（Deep Learning Scientist）

教程同時強調圖形（Graphics）在數學、統計與機器學習中都扮演重要角色（S45「Graphics」）。

3.2 線性函數（Linear Functions）

三句口訣（S45、S46 開首）：**Linear 意指「直」**；線性函數是一條直線；線性圖代表一個線性函數。

3.2.1 函數的概念

函數（Function）是一種特殊的關係：**每個輸入都有一個輸出**。函數常寫成 f(x)，其中 x 是輸入（S46「Linear Functions」）。

f(x) = x 的結果

x	y	y = x
1	1	y = x = 1
2	2	y = x = 2
3	3	y = x = 3
4	4	y = x = 4
5	5	y = x = 5

f(x) = 2x 的結果

x	y	y = 2x
1	2	y = 2x = 2
2	4	y = 2x = 4
3	6	y = 2x = 6
4	8	y = 2x = 8
5	10	y = 2x = 10

3.2.2 線性方程與非線性方程

線性方程（Linear Equation）是直線的方程。教程的例子（S46「Linear Equations」）：

- y = x
- y = x*2
- y = x*2 + 7
- y = ax + b
- 5x = 3y
- y/2 = 6

線性方程**不能**包含指數或平方根。以下是非線性的例子（S46「Non-Linear Equations」，以 JavaScript 寫法表示）：

- `y = x**2`（x 的平方）
- `y = Math.sqrt(x)`（平方根）
- `y = Math.sin(x)`（正弦函數）

3.2.3 線性迴歸與線性最小二乘法

- **線性迴歸（Linear Regression）**嘗試以一條線性圖擬合數據，從而模擬兩個變數之間的關係：一個變數（x）視為數據，另一個（y）視為因變數。例如用線性迴歸把樓價與面積聯繫起來（S46「Linear Regression」）。
- 線性代數用於求解線性方程；**線性最小二乘法（Linear Least Squares, LLS）**是一組用以解決線性迴歸中統計問題的公式（S46「Linear Least Squares」）。

3.3 線性圖（Linear Graphs）：斜率與截距

機器學習常用線形圖顯示關係。線形圖顯示線性函數 **y = ax + b** 的值。三個關鍵詞（S06 開首）：

關鍵詞	意義	在方程中的位置
線性（Linear）	直的；線性圖是一條直線。圖有兩條軸：x 軸（水平）與 y 軸（垂直）。	—
斜率（Slope）	圖的角度（angle）。	y = ax 中的 **a** 值

關鍵詞	意義	在方程中的位置
截距（Intercept）	圖的起始值（start value）。	y = ax + b 中的 **b** 值

教程用 Plotly.js 繪製三個遞進的例子（S06「Linear」「Slope」「Intercept」）：先畫 y = x；再加入斜率 1.2（y = 1.2x）；最後加入截距 7（y = 1.2x + 7）。以下是最完整的第三個例子：

S06「Intercept」範例：斜率 1.2、截距 7

```
let slope = 1.2;
let intercept = 7;
const xValues = [];
const yValues = [];

// Generate values
for (let x = 0; x <= 10; x += 1) {
  xValues.push(x);
  yValues.push(x * slope + intercept);
}

// Define Data
const data = [{
  x: xValues,
  y: yValues,
  mode: "lines"
}];

// Define Layout
const layout = {title: "Slope=" + slope + " Intercept=" + intercept};

// Display using Plotly
Plotly.newPlot("myPlot", data, layout);
```

> **【學習提示】** 理解斜率與截距是後續內容的基礎：在第 5.3 節，機器學習把同一條直線寫成 y = b + wx，其中斜率 a 改稱**權重（weight, w）**，截距 b 改稱**偏置（bias）**；第 7 節的梯度下降正是不斷調整 w 與 b 來尋找最佳直線。

3.4 線性代數（Linear Algebra）概覽

教程說「機器學習專家離不開線性代數」，因為 ML 大量使用**純量（Scalars）**、**向量（Vectors）**、**矩陣（Matrices）**與**張量（Tensors）**；S47 的目的是突出線性代數中用於機器學習與深度學習等數據科學項目的部分。教程又說「向量與矩陣是數據的語言」，「有了向量與矩陣，你可以發現秘密（Discover Secrets）」（S47「Vectors and Matrices」）。

物件	定義（S47）	JavaScript 寫法（S47、S49、S50）	張量觀點（S50）
純量（Scalar）	單一個數字	`const myScalar = 1;`	0 維張量；0 條軸；秩（Rank）0
向量（Vector）	一個數字陣列（1 維陣列）	`const myArray = [50,60,70,...,150];`（長度 11）或寫成單欄矩陣 `[[1],[2],[3]]`	1 維張量；1 條軸；秩 1
矩陣（Matrix）	2 維陣列（陣列的陣列：每個內層陣列是一列，內層元素是欄）	`const myArray = [[1,2],[3,4],[5,6]];`	2 維張量；2 條軸；秩 2
張量（Tensor）	N 維矩陣；向量與矩陣向更高維度的推廣	具有多個索引的陣列，例如 `[[1,2,3],[4,5,6],[7,8,9]]`	一般指維度大於 2（R > 2）的「真正張量」

教程亦引用 Wikipedia 的定義：線性代數是關於線性方程（及線性映射）及其在向量空間與矩陣中的表示的數學分支，是幾乎所有數學領域的核心（S47 末段）。

> **【來源問題／不一致之處】** S45、S47 與 S50 都載有同一幅「Scalar／Vector(s)／Matrix／Tensor」示意表格（以 1 2 3 … 排列成不同形狀）。該表格在 HTML 中以巢狀表格呈現，純文字擷取後難以還原，講義以上表重新整理。

3.5 向量（Vectors）

向量是 1 維陣列；向量有**大小（Magnitude）**與**方向（Direction）**；向量通常描述**運動（Motion）**或**力（Force）**（S48 開首）。

3.5.1 向量記法與幾何意義

向量可以寫成橫排 v = [1 2 3]，或寫成直排（一欄三列）（S48「Vector Notation」）。在幾何上，向量以箭頭表示：**長度**代表大小，**箭頭**代表方向（S48「Vectors in Geometry」）。

3.5.2 運動、加法與減法

- **運動**：向量是運動的基本構件；在幾何中，向量可描述由一點到另一點的移動。向量 [3, 2] 的意思是「向右 3、向上 2」（S48「Motion」）。
- **向量加法**：求 a + b 時，把向量 b 移動至其尾端接上向量 a 的頭端（這不會改變 b），然後由 a 的尾端到 b 的頭端的線便是 a + b（S48「Vector Addition」）。
- **向量減法**：向量 −a 是 +a 的相反；a 與 −a 大小相同、方向相反（S48「Vector Subtraction」）。

3.5.3 純量運算

向量可以透過把一個純量加到、減去或乘以向量的所有元素而改變（S48「Scalar Operations」）：

a = [1 1 1]；a + 1 = [2 2 2]；[1 2 3] + 1 = [2 3 4]

[2 2 2] × 3 = [6 6 6]；[6 6 6] ÷ 3 = [2 2 2]

教程指出向量乘法與普通乘法有許多相同性質；並以**力（Force）**與**速度（Velocity）**為例：兩者都是具有大小與方向的向量（S48「Force」「Velocity」）。

3.6 矩陣（Matrices）

矩陣是一組數字，排列成**矩形陣列**，以**列（Rows）與欄（Columns）**組織（S49 開首）。

3.6.1 矩陣的維度

矩陣的維度以「列數 × 欄數」表示（S49「Matrix Dimensions」）：

矩陣	維度
[2 5 3]（1 列 3 欄）	1×3
[2 5 3; 4 7 1]（2 列 3 欄）	2×3

> **【補充說明（非來源內容）】** 講義以分號「;」分隔矩陣的列，例如 [2 5 3; 4 7 1] 表示第一列是 2 5 3，第二列是 4 7 1。

3.6.2 特殊矩陣

類型	定義（S49）	教程例子
方陣（Square Matrix）	列數與欄數相同；n×n 矩陣稱為 n 階方陣。	2 階：[1 2; 3 4]；4 階：[1 −2 3 4; 5 6 −7 8; 4 3 2 −1; 8 7 6 −5]
對角矩陣（Diagonal Matrix）	只有對角線位置有值，其餘為零。	[2 0 0; 0 5 0; 0 0 3]
純量矩陣（Scalar Matrix）	對角線元素相等，其餘為零。	[3 0 0 0; 0 3 0 0; 0 0 3 0; 0 0 0 3]
單位矩陣（Identity Matrix, I）	對角線為 1，其餘為 0；是矩陣世界中「1」的等價物。**任何矩陣乘以單位矩陣，結果等於原矩陣。**	I = [1 0 0 0; 0 1 0 0; 0 0 1 0; 0 0 0 1]
零矩陣（Zero／Null Matrix）	全部元素為零。	[0 0 0; 0 0 0]
相等矩陣（Equal Matrices）	每個對應元素都相同。	[2 5 3; 4 7 1] = [2 5 3; 4 7 1]
負矩陣（Negative Matrix）	把每個元素變號。	−[−2 5 3; −4 7 1] = [2 −5 −3; 4 −7 −1]

3.6.3 JavaScript 中的矩陣與 math.js

教程把線性代數物件對應到 JavaScript（S49「Linear Algebra in JavaScript」）：純量是 `const scalar = 1;`，陣列是 `const array = [1, 2, 3];`，矩陣是 2 維陣列 `const matrix = [[1,2],[3,4],[5,6]];`，向量可寫成單欄矩陣 `[[1],[2],[3]]` 或普通陣列 `[1, 2, 3]`。

用純 JavaScript 編寫矩陣運算「很容易變成一堆意粉般的迴圈」，使用程式庫可省卻許多麻煩。最常用的矩陣運算程式庫之一是 **math.js**，只需一行即可加入網頁（S49「JavaScript Matrix Operations」）：

加入 math.js

```
<script src="https://cdnjs.cloudflare.com/ajax/libs/mathjs/9.3.2/math.js"></script>
```

3.6.4 矩陣加法、減法與純量乘除

只有維度相同的矩陣才能相加或相減，方法是對應元素逐一相加／相減（S49「Adding Matrices」「Subtracting Matrices」）：

$$[2\ 5\ 3;\ 4\ 7\ 1] + [4\ 7\ 1;\ 2\ 5\ 3] = [6\ 12\ 4;\ 6\ 12\ 4]$$

$$[2\ 5\ 3;\ 4\ 7\ 1] - [4\ 7\ 1;\ 2\ 5\ 3] = [-2\ -2\ 2;\ 2\ 2\ -2]$$

math.js 矩陣加減

```
const mA = math.matrix([[1, 2], [3, 4], [5, 6]]);
const mB = math.matrix([[1,-1], [2,-2], [3,-3]]);

// Matrix Addition
const matrixAdd = math.add(mA, mB);
// Result [ [2, 1], [5, 2], [8, 3] ]

// Matrix Subtraction
const matrixSub = math.subtract(mA, mB);
// Result [ [0, 3], [1, 6], [2, 9] ]
```

純量乘法：把矩陣中每個數字乘以該純量即可（S49「Scalar Multiplication」）：

$$[2\ 5\ 3;\ 4\ 7\ 1] \times 2 = [4\ 10\ 6;\ 8\ 14\ 2]$$

math.js 純量乘法與除法

```
const mA = math.matrix([[1, 2], [3, 4], [5, 6]]);
const matrixMult = math.multiply(2, mA);
// Result [ [2, 4], [6, 8], [10, 12] ]

const mC = math.matrix([[0, 2], [4, 6], [8, 10]]);
const matrixDiv = math.divide(mC, 2);
// Result [ [0, 1], [2, 3], [4, 5] ]
```

3.6.5 轉置（Transpose）

轉置矩陣是把列換成欄；交換列與欄等於把矩陣沿其對角線旋轉（S49「Transpose a Matrix」）：

$$A = [1\ 2; 3\ 4] \quad \Rightarrow \quad A^T = [1\ 3; 2\ 4]$$

3.6.6 矩陣乘法（Multiplying Matrices）

矩陣乘法較為困難。**只有當矩陣 A 的欄數等於矩陣 B 的列數時，兩者才能相乘**；然後需要計算「點積（dot product）」——把 A 的數字與 B 中對應的數字相乘後再相加（S49「Multiplying Matrices」）。教程例子：

math.js 矩陣乘法

```
const mA = math.matrix([1, 2, 3]);
const mB = math.matrix([[1, 4, 7], [2, 5, 8], [3, 6, 9]]);

// Matrix Multiplication
const matrixMult = math.multiply(mA, mB);
// Result [14, 32, 50]
```

S49「Explained」：A 與 B 每一欄的點積

計算	結果
(1,2,3) ・ (1,2,3) = 1×1 + 2×2 + 3×3 =	14
(1,2,3) ・ (4,5,6) = 1×4 + 2×5 + 3×6 =	32
(1,2,3) ・ (7,8,9) = 1×7 + 2×8 + 3×9 =	50

教程說：「如果你懂得矩陣乘法，就能解決許多複雜的方程。」並給出一個賣玫瑰的例子（S49「Example」）：

- 紅玫瑰每枝 $3、白玫瑰每枝 $4、黃玫瑰每枝 $2。
- 星期一賣出 260 枝、星期二 200 枝、星期三 120 枝。問：全部銷售額是多少？

	$3（紅）	$4（白）	$2（黃）
星期一	120	80	60
星期二	90	70	40
星期三	60	40	20

玫瑰銷售額：價格向量 × 銷量矩陣

```
const mA = math.matrix([3, 4, 2]);
const mB = math.matrix([[120, 90, 60], [80, 70, 40], [60, 40, 20]]);

// Matrix Multiplication
const matrixMult = math.multiply(mA, mB);
// Result [800, 630, 380]
```

計算		結果
(3,4,2) · (120,80,60)	= 3×120 + 4×80 + 2×60	= $800（星期一）
(3,4,2) · (90,70,40)	= 3×90 + 4×70 + 2×40	= $630（星期二）
(3,4,2) · (60,40,20)	= 3×60 + 4×40 + 2×20	= $380（星期三）
合計		**$1,810**

> **【來源問題／不一致之處】** 注意 mB 的排列：每一**欄**代表一天（星期一、二、三），每一**列**代表一種玫瑰（紅、白、黃），因此價格列向量 (3,4,2) 與 mB 的每一欄作點積，便得出該天的銷售額。S49 原始程式碼中 mB 的結尾少了一個右方括號（`[60, 40, 20]);`），講義已補正為 `[60, 40, 20]]);`。

3.6.7 矩陣分解

教程指出：在 AI 中需要懂得如何分解矩陣；**矩陣分解（Matrix Factorization）**是線性代數的關鍵工具，尤其用於線性最小二乘法（S49「Matrix Factorization」）。教程沒有進一步展開。

3.7 張量（Tensors）

張量是 N 維矩陣，是向量與矩陣向更高維度的推廣（S50 開首）：純量是 0 維張量、向量是 1 維張量、矩陣是 2 維張量。

3.7.1 張量的秩（Rank）

張量在 N 維空間中可以擁有的「方向」數目稱為張量的**秩（Rank）**，記作 R（S50「Tensor Ranks」）：

物件	軸數	秩	維度
純量（單一數字）	0	0	0 維張量
向量（數字陣列）	1	1	1 維張量
矩陣（2 維陣列）	2	2	2 維張量

嚴格來說以上全都是張量，但一般談到「張量」時，通常指維度大於 2（R > 2）的矩陣（S50「Real Tensors」）。

3.7.2 JavaScript 中的張量運算：TensorFlow.js

與矩陣一樣，用純 JavaScript 編寫張量運算容易變成一堆迴圈；最常用的張量運算程式庫之一是 **TensorFlow.js**（S50「JavaScript Tensor Operations」）。以下範例在 S20 亦有出現，詳見第 9.3 節：

TensorFlow.js 張量加減

```
const tensorA = tf.tensor([[1, 2], [3, 4], [5, 6]]);
const tensorB = tf.tensor([[1,-1], [2,-2], [3,-3]]);

// Tensor Addition
const tensorAdd = tensorA.add(tensorB);   // [ [2, 1], [5, 2], [8, 3] ]

// Tensor Subtraction
const tensorSub = tensorA.sub(tensorB);   // [ [0, 3], [1, 6], [2, 9] ]
```

4. 統計與概率基礎

統計是用來回答有關數據問題的工具：甚麼是常見的（Common）？甚麼是預期的（Expected）？甚麼是正常的（Normal）？概率是多少（Probability）？（S51 開首；S45「Statistics」亦指統計是關於收集、分析、詮釋與呈現數據。）

4.1 推論統計與描述統計

類型	定義（S51）	教程例子
推論統計（Inferential Statistics）	從一個小**樣本**量化**總體（population）**性質的方法：取樣本數據，對整個總體作出預測。	在商店問 100 名顧客是否喜歡朱古力，再推斷 91% 的顧客喜歡朱古力。教程的「朱古力趣聞」：十人中有九人愛朱古力；50% 美國人口每天不能沒有朱古力。
描述統計（Descriptive Statistics）	總結（描述）一組數據的觀察結果。	由於每名初生嬰兒都有登記，我們知道每 100 名中有 51 名是男孩，因而可預測新生兒是男孩的機會是 51%。教程稱這個比例不是 50%「是一個謎」，只知道自 17 世紀以來一直如此傾斜。

> **【來源重點】** 原始觀察只是數據，不是真正的知識；描述統計把原始觀察轉化成可理解的數據（S51「Note」）。

描述統計分為兩大類量度（S51「Descriptive Statistics Measurements」、S52 開首、S53 開首）：

類別	量度
集中趨勢（Tendency，中心量度）	平均數（Mean，平均值）、中位數（Median，中點值）、眾數（Mode，最常見的值）
離散程度（Spread／Variability）	最小值與最大值（Min and Max）、標準差（Standard Deviation）、方差（Variance）、偏度（Skewness）、峰度（Kurtosis）；S53 另列出「分佈（Distribution）」

4.2 集中趨勢：平均數、中位數、眾數

4.2.1 平均數（The Mean）

平均數是所有數值的平均：**把所有數值相加，再除以數值的個數**（「總和除以個數」）（S52「The Mean」）。教程以 11 個樓價數值為例：

7	**8**	**8**	**9**	**9**	**9**	**10**	**11**	**14**	**14**	**15**

$$(7+8+8+9+9+9+10+11+14+14+15) \div 11 = 114 \div 11 = 10.3636363636\cdots$$

計算平均數

```
let mean = (7+8+8+9+9+9+10+11+14+14+15)/11;

// 或使用 math.js
const values = [7,8,8,9,9,9,10,11,14,14,15];
let mean = math.mean(values);
```

4.2.2 中位數（The Median）

中位數是數值**排序後**位於中間的值（S52「The Median」）。教程以 13 個車速數值為例：

奇數個數值的中位數

```
原始：99,86,87,88,111,86,103,87,94,78,77,85,86
排序：77,78,85,86,86,86,[87],87,88,94,99,103,111   → 中位數 = 87

const speed = [99,86,87,88,111,86,103,87,94,78,77,85,86];
let median = math.median(speed);
```

若中間有兩個數，則把兩者之和除以二：

偶數個數值的中位數

```
排序：77,78,85,86,86,[86,87],87,88,94,99,103   → (86 + 87) / 2 = 86.5

const speed = [99,86,87,88,86,103,87,94,78,77,85,86];
let median = math.median(speed);
```

4.2.3 眾數（The Mode）

眾數是出現次數最多的值。在 99, **86**, 87, 88, 111, **86**, 103, 87, 94, 78, 77, 85, **86** 中，86 出現三次，故眾數為 86（S52「The Mode」）。

計算眾數

```
const speed = [99,86,87,88,86,103,87,94,78,77,85,86];
let mode = math.mode(speed);
```

4.2.4 離群值（Outliers）

離群值是位於其他數值「之外」的值，例如 99, 86, 87, 88, 111, 86, 103, 87, 94, 78, **300**, 85, 86 中的 300。**離群值可以大幅改變平均數**；有時我們不使用它們（可能是錯誤），或改用中位數或眾數（S52「Outliers」）。

含離群值的平均數

```
const values = [99,86,87,88,111,86,103,87,94,78,300,85,86];
```

```
let mean = math.mean(values);
```

> **【補充說明（非來源內容）】** 不含 300 時（以 77 代替）13 個車速的平均約為 89.8；把 77 換成 300 後平均升至約 106.9，而中位數仍是 87——這正是「離群值大幅改變平均數、中位數較穩健」的具體示範（數值由講義編者計算）。

4.3 離散程度：方差與標準差

4.3.1 方差（Variance）

在統計學中，方差是**各數值與平均數之差的平方的平均**；換言之，方差描述一組數字相對於平均值的**分散程度**（S53「The Variance」）。計算步驟（以同一組 11 個樓價數值為例）：

1. 計算平均數 m（= 10.3636…）。
2. 計算每個數值與 m 之差的平方，並全部相加，得「平方和（Sum of Squares, ss）」。
3. 把 ss 除以數值個數（11），得方差。

計算方差（講義已修正原文的項數，見下方說明）

```
// Calculate the Mean (m)
let m = (7+8+8+9+9+9+10+11+14+14+15)/11;

// Calculate the Sum of Squares (ss)
let ss = (7-m)**2 + (8-m)**2 + (8-m)**2 + (9-m)**2 + (9-m)**2 + (9-m)**2
       + (10-m)**2 + (11-m)**2 + (14-m)**2 + (14-m)**2 + (15-m)**2;

// Calculate the Variance
let variance = ss / 11;

// 或使用 math.js（"uncorrected" 表示除以 n 而非 n-1）
const values = [7,8,8,9,9,9,10,11,14,14,15];
let variance = math.variance(values, "uncorrected");
```

> **【來源問題／不一致之處】** S53 原始的平方和程式碼列出了**四個** $(9-m)^2$ 與**一個** $(14-m)^2$，但數據中 9 出現三次、14 出現兩次，與表格數據不符；講義上方程式碼已按表格數據修正為三個 $(9-m)^2$ 與兩個 $(14-m)^2$。其後「Standard Deviation」小節的程式碼更把平均數寫成 (7+8+8+9+9+9+10+11+14+15)/11（少了一個 14，只有 10 個數卻除以 11），而 math.js 的 std 範例則用了 [7,8,8,9,9,9,9,10,11,14,15]（四個 9、一個 14）。這三處都是來源的錯誤。按正確數據計算：**方差（除以 n）≈ 6.9587，標準差 ≈ 2.638**；若用樣本方差（除以 n−1）則約為 7.6545，標準差約 2.767（數值由講義編者計算）。

4.3.2 標準差（Standard Deviation）

標準差量度數字的分散程度，符號是 **σ**（希臘字母 sigma），公式是**方差的平方根**（√variance）（S53「Standard Deviation」）。教程形容「偏差（Deviation）是一種距離的量度」——所有數值平均而言距離平均數（中間）有多遠。

計算標準差

```
// ...接續上面計算出 variance 之後
let std = Math.sqrt(variance);

// 或使用 math.js
const values = [7,8,8,9,9,9,10,11,14,14,15];
let std = math.std(values, "uncorrected");
```

> **【補充說明（非來源內容）】** math.js 的 `variance()` 與 `std()` 預設使用 “unbiased”（除以 n−1，即樣本方差）；加上第二個參數 `"uncorrected"` 才會除以 n，與教程手動計算的「平均平方差」一致。

4.4 分佈（Distribution）

4.4.1 正態分佈（Normal Distribution）

正態分佈曲線是一條**鐘形曲線（bell-shaped curve）**。曲線上每一條帶的闊度是距離平均值 1 個標準差（S54「Normal Distribution」）：

與平均值的距離	佔總體的百分比	教程的詮釋
1 個標準差以內	68.27%	大部分觀察值在 1 個標準差以內
2 個標準差以內	95.45%	幾乎所有觀察值在 2 個標準差以內
3 個標準差以內	99.73%	實際上全部觀察值在 3 個標準差以內

正態分佈的事實（S54「Normal Distribution Facts」）：

- 正態分佈是**對稱**的：峰頂永遠把分佈一分為二。
- 正態分佈是一種**概率分佈**。
- 許多觀察值都服從正態分佈：你的智商、體重、身高、薪金、血壓。
- 正態分佈顯示接近平均值的數值比遠離平均值的數值更頻繁出現。
- **68–95–99.7 法則**（又稱**經驗法則 Empirical Rule**）是記住正態分佈各帶百分比的速記法。
- 正態分佈又稱**高斯分佈（Gaussian Distribution）**與**鐘形曲線（Bell Curve）**。

4.4.2 誤差幅度（Margin of Error）

統計學家總會嘗試以 100% 準確度預測一切，但總會有一些不確定性。**誤差幅度**是量化這種統計不確定性的數字；不同的幅度界定了我們相信正確答案所在的不同範圍。可接受的幅度是判斷問題，並取決於答案有多重要。**收集的樣本越多，誤差幅度越低**（S54「The Margin of Error」）。

如何詮釋（S54「How to Interpret Margin of Error」）：假設抽樣總體中 55% 的人表示會投「贊成」票。推算到整個總體時，加減誤差幅度便得出可能結果的範圍：

- 誤差幅度 3%：有信心 52% 至 58% 的人會投「贊成」。
- 誤差幅度 10%：有信心 45% 至 65% 的人會投「贊成」。

4.4.3 偏度（Skewness）與峰度（Kurtosis）

- **偏度**是相對於鐘形曲線（正態分佈）的一種扭曲（不對稱）（S54「Skewness」）。
- **峰度**也是相對於正態分佈的一種扭曲；偏度描述**一側**尾部出現的非預期數值，峰度則描述**兩側**尾部的非預期數值。教程以兩幅圖示說明：負峰度（比正態分佈更低平）與正峰度（比正態分佈更高尖）（S54「Kurtosis」）。

4.5 概率（Probability）

概率是關於某事**有多可能發生**，或某事有多可能為真。數學上的概率是 **0 與 1 之間的數字**：0 表示不可能，1 表示必然（S55 開首）。事件的概率是（S55「The Probability of an Event」）：

概率 = 事件可以發生的方式數 ÷ 所有可能結果數　（Probability = # of Ways / Outcomes）

4.5.1 擲硬幣

擲一枚硬幣有兩個可能結果（S55「Tossing Coins」）：正面（Heads）1/2 = 0.5；反面（Tails）1/2 = 0.5。事件 A 的概率常寫作 **P(A)**。擲兩枚硬幣有 4 個可能結果（S55「P(A) - The Probability」）：

事件	P(A)
正面 + 正面	1/4 = 0.25
反面 + 反面	1/4 = 0.25
正面 + 反面	1/4 = 0.25
反面 + 正面	1/4 = 0.25

4.5.2 擲骰子

擲一粒骰子有 6 個可能結果，每個點數的概率都是 1/6 = 0.166666（S55「Throwing Dices」）。

- 同時擲出三個 4 的可能性是 $(1/6)^3$：`let p = Math.pow(1/6, 3);`
- 同時擲出三個相同點數（任何點數）的可能性是前者的 6 倍（擲出 1 + 擲出 2 + … + 擲出 6）：`let p = Math.pow(1/6, 3) * 6;`

> **【補充說明（非來源內容）】** $(1/6)^3 = 1/216 \approx 0.00463$；乘以 6 得 $6/216 = 1/36 \approx 0.0278$（數值由講義編者計算）。

4.5.3 六個球

袋中有 6 個球：3 紅、2 綠、1 藍。蒙眼抽一個，抽到綠球的概率是多少？可以發生的方式有 2 種（2 個綠球），可能結果有 6 種（6 個球），故 P(green) = 2/6 = 0.333333（S45「Probability」、S55「6 Balls」）。

P(A)	方式／結果	概率
P(red)	3/6	0.5000000
P(green)	2/6	0.3333333
P(blue)	1/6	0.1666666

S55「P(A) = P(B)」

比較	意義
P(A) = P(B)	事件 A 與 B 發生的機會相同
P(A) > P(B)	事件 A 發生的機會較高
P(A) < P(B)	事件 A 發生的機會較低

套用到六個球：P(red) > P(green)、P(red) > P(blue)、P(green) > P(blue)（抽到紅比綠更可能、紅比藍更可能、綠比藍更可能）；反過來 P(blue) < P(green)、P(blue) < P(red)、P(green) < P(red)（S55）。

4.5.4 抽到國王牌

從一副 52 張啤牌中抽到國王（King）的概率是 4/52：可以發生的方式有 4 種（4 張 K），可能結果有 52 種（52 張牌），故 P(king) = 4/52 = 0.076923（S55「Choosing a King」）。

5. 機器學習的核心概念：數據、術語、學習類型、聚類與迴歸

5.1 數據（Data）

教程指出，一個機器學習項目**高達 80%** 的工作都與收集數據有關，要回答：需要甚麼數據？有甚麼數據可用？如何選擇、收集、清理、準備與使用數據？（S14 開首）

5.1.1 甚麼是數據？

在機器學習中，數據是**事實的集合**（S14「What is Data?」）：

類型	例子
數字（Numbers）	價格、日期
量度（Measurements）	尺寸、高度、重量
詞語（Words）	名字與地方
觀察（Observations）	數汽車
描述（Descriptions）	「天氣很冷」

智能需要數據：地產經紀需要已售樓宇的數據來估價；機器學習程式同樣需要數據來估價。數據幫助我們看見和理解、發現新機會、化解誤解（S14「Intelligence Needs Data」）。教程舉出三個行業例子（S14）：

- **醫療**：醫療與生命科學收集公共衞生數據與病人數據，以學習如何改善病人護理、拯救生命。
- **商業**：許多行業中最成功的公司都是數據驅動的，用精密的數據分析學習如何做得更好。
- **金融**：銀行與保險公司收集並評估有關客戶、貸款與存款的數據，以支援策略決策。

5.1.2 儲存數據：樓價與面積數據集

最常收集的數據是數字與量度，通常以**陣列**儲存，代表數值之間的關係。教程貫穿全書使用以下「樓價對面積」數據（S14「Storing Data」；S07「Data Collection」相同）：

面積（Size，平方米）	50	60	70	80	90	100	110	120	130	140	150
價格（Price，百萬）	7	8	8	9	9	9	10	11	14	14	15

> **【學習提示】** 這組 11 對數據在教程多處重複使用：散點圖（S07）、迴歸（S16）、Canvas／Plotly／Chart.js／Google Chart 範例（S32–S35），以及第 4 節統計計算中的 11 個價格值（S52、S53）。記住它，便能貫通多個章節。

5.1.3 定量與定性；普查與抽樣

概念	定義（S14）	例子
定量數據（Quantitative）	數值型的	55 輛車、15 米、35 個小孩
定性數據（Qualitative）	描述型的	很冷、很長、很有趣
普查（Census）	為群組中**每一個**成員收集數據；準確但難以執行	問全美國每一個人是否吸煙
抽樣（Sample）	為群組中**部分**成員收集數據；不準確但較易執行	問 10,000 人
總體（Population）	我們想從中收集資訊的一群個體（物件）	—
隨機樣本（Random Sample）	總體中每個成員被抽中的機會均等；樣本必須隨機收集才能代表總體	—
抽樣偏差（Sampling Bias／Error）	收集樣本的方式令某些個體被納入的可能性較低（或較高）	—

5.1.4 大數據與數據挖掘

- **大數據（Big Data）**是人類在沒有先進機器協助下無法處理的數據。大數據沒有以大小界定的定義，但隨着我們持續收集更多數據並以越來越低的成本儲存，數據集正變得越來越大（S14「Big Data」）。
- **數據挖掘（Data Mining）**：大數據伴隨着複雜的數據結構；大數據處理的很大部分是**提煉（refining）數據**（S14「Data Mining」）。

5.2 散點圖與數據關係

收集數據是任何機器智能項目最重要的部分（S07「Data Collection」）。**散點圖（Scatter Plot）**是把點散佈在一個區域上，表示兩個數值之間的關係（S07「Scatter Plots」）。以下是以 Plotly.js 繪製樓價數據的範例：

S07 散點圖（把 mode 改為 "lines" 便成為線形圖）

```
const xArray = [50,60,70,80,90,100,110,120,130,140,150];
const yArray = [7,8,8,9,9,9,10,11,14,14,15];

// Define Data
const data = [{
  x: xArray,
  y: yArray,
  mode:"markers"
}];

// Define Layout
const layout = {
  xaxis: {range: [40, 160], title: "Square Meters"},
  yaxis: {range: [5, 16], title: "Price in Millions"},
  title: "House Prices vs. Size"
```

```
};

// Display with Plotly
Plotly.newPlot("myPlot", data, layout);
```

散點圖的用途（S07「When to Use Scatter Plots」）：

- 看見「全局（Big Picture）」
- 比較不同數值
- 發現潛在趨勢
- 發現數據中的模式
- 發現數據之間的關係
- 發現聚類（Clusters）與相關性（Correlations）

5.3 機器學習術語：關係、標籤、特徵、模型、訓練與推論

5.3.1 關係（Relationships）

機器學習系統利用輸入之間的**關係**來產生**預測**。代數與機器學習對同一條直線有不同寫法（S13「Machine Learning Relationships」）：

	代數（Algebra）	**機器學習**
方程	y = ax + b	y = b + wx
y	因變數	我們想預測的**標籤（label）**
a／w	直線的斜率	**權重（weight）**（即斜率）
x	輸入值	**特徵（features）**（輸入值）
b	截距	截距（在後續章節稱為偏置 bias）

有時會有多個特徵（輸入值），各有不同權重（S13「Machine Learning Features」）：

$$y = b + w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4$$

5.3.2 標籤與特徵

- **標籤（Label）**：我們想預測的東西，就像線性圖中的 y（S13「Machine Learning Labels」）。
- **特徵（Features）**：輸入，就像線性圖中的 x 值（S13「Machine Learning Features」）。

5.3.3 模型、訓練、推論與兩個階段

- **模型（Model）**定義標籤（y）與特徵（x）之間的關係。模型的生命週期有三個階段：**數據收集（Data Collection）**、**訓練（Training）**、**推論（Inference）**（S13「Machine Learning Models」）。
- **訓練**的目標是建立一個能回答問題的模型，例如「一間樓的預期價格是多少？」（S13「Machine Learning Training」）

- **推論**是用已訓練的模型以即時數據推斷（預測）數值，就像把模型投入生產（S13「Machine Learning Inference」）。

階段	說明（S13「Machine Learning Phases」）
1. 訓練（Training）	用輸入數據計算模型的參數。
2. 推論（Inference）	「已訓練」的模型對任何輸入輸出正確的數據。

5.4 學習類型

類型	定義與特點（S13）	例子（S13）
監督式學習（Supervised Learning）	用一組輸入變數預測輸出變數的值；使用**有標籤的數據**（已知答案的數據）訓練演算法去**分類數據**與**預測結果**。	根據已知的垃圾郵件例子分類「甚麼是垃圾郵件」；根據你看過的影片預測你喜歡哪類影片。
非監督式學習（Unsupervised Learning）	利用**無標籤數據集**中的模式，嘗試理解數據中的模式（或分組）；用於預測未定義的關係（如數據中有意義的模式）；關乎建立能自我改進的演算法。教程預期機器學習將轉向非監督式學習，讓程式員不必建立模型便能解決問題。	聚類（見 5.5 節）
強化學習（Reinforcement Learning）	以非監督式學習為基礎，但會從用戶接收「決定是好是壞」的回饋；回饋有助改進模型。	—
自監督學習（Self-Supervised Learning）	與非監督式學習相似，同樣處理沒有人工標籤的數據；分別在於非監督式學習使用聚類、分組與降維，而自監督學習會為迴歸與分類任務**自行得出結論**。	—

> **【補充說明（非來源內容）】** 教程把強化學習描述為「以非監督式學習為基礎」。在較正式的分類中，強化學習通常被視為與監督式、非監督式並列的第三大範式：代理人（agent）透過與環境互動獲得獎勵訊號來學習策略，而不是依賴用戶回饋。此屬補充觀點，讀者可自行進一步查閱。

5.5 聚類（Clustering）與相關系數

5.5.1 聚類的概念

- **聚類（Clusters）**是基於相似性的數據集合；在圖中聚集在一起的數據點往往可以被歸入不同的聚類。教程的示意圖可分辨出 3 個聚類（S15「Clusters」）。
- 聚類蘊含大量有價值的資訊，但形狀多樣，辨認聚類的兩個主要方法是：**視覺化**與**聚類演算法**（S15「Identifying Clusters」）。

- 聚類是一種**非監督式學習**，目標是把相似的數據收集到同一組，把不相似的數據收集到其他組（S15「Clustering」）。

方法	原理（S15「Clustering Methods」）	常見演算法
密度法（Density Method）	認為密集區域中的點之間有較多相似性與差異性（相對於低密度區域的點）；準確度高，並有合併聚類的能力。	DBSCAN、OPTICS
層次法（Hierarchical Method）	以樹狀結構形成聚類；新聚類由先前形成的聚類組成。	CURE、BIRCH
網格法（Grid-based Method）	把數據編排成有限數目的格子，形成網格結構。	CLIQUE、STING
分割法（Partitioning Method）	把物件分割成 k 個聚類，每個分割形成一個聚類。	CLARANS

5.5.2 相關系數（Correlation Coefficient, r）

相關系數 r 描述散點圖上 x／y 變數之間**線性關係的強度與方向**，其值永遠介乎 −1 與 +1 之間（S15「Correlation Coefficient」）：

r 值	形容	關係
−1.00	完美下坡（Perfect downhill）	負線性關係
−0.70	強下坡（Strong downhill）	負線性關係
−0.50	中等下坡（Moderate downhill）	負線性關係
−0.30	弱下坡（Weak downhill）	負線性關係
0	—	沒有線性關係
+0.30	弱上坡（Weak uphill）	正線性關係
+0.50	中等上坡（Moderate uphill）	正線性關係
+0.70	強上坡（Strong uphill）	正線性關係
+1.00	完美上坡（Perfect uphill）	正線性關係

教程以四幅散點圖示例：完美上坡 +1.00、完美下坡 −1.00、強上坡 +0.61、沒有關係（S15）。

5.6 線性迴歸（Linear Regressions）

迴歸（Regression）是判定一個變數（y）與其他變數（x）之間關係的方法。在統計學中，線性迴歸是對 y 與 x 之間線性關係建模的方法；在機器學習中，線性迴歸是一種**監督式**機器學習演算法（S16 開首）。

5.6.1 如何從散點預測未來價格？

面對樓價散點圖，教程提出三種預測方法（S16「Predicting Values」）：

1. 手繪線性圖（Use hand drawn linear graph）
2. 建立線性關係模型（Model a linear relationship）
3. 建立線性迴歸模型（Model a linear regression）

方法一：以最低與最高價格畫一條直線

教程直接把 (50, 7) 與 (150, 15) 連成一條線（S16「Linear Graphs」）：

S16「Linear Graphs」的關鍵部分

```
const data = [
  {x:xArray, y:yArray, mode:"markers"},
  {x:[50,150], y:[7,15], mode:"line"}
];
```

> **【來源問題／不一致之處】** S16 這個範例的 yArray 寫作 [7,8,8,9,9,9,9,10,11,14,14,15]，共 **12 個**值，而 xArray 只有 11 個值（多了一個 9），與教程其他地方使用的 11 個價格值不符，應屬筆誤。

方法二：以平均斜率建立線性關係

這個模型以「y 總和 ÷ x 總和」作為斜率，截距為 0（S16「Linear Relationships」）：

S16「Linear Relationships」

```
// Calculate Slope
let xSum = xArray.reduce(function(a, b){return a + b;}, 0);
let ySum = yArray.reduce(function(a, b){return a + b;}, 0);
let slope = ySum / xSum;

// Generate values
const xValues = [];
const yValues = [];
for (let x = 50; x <= 150; x += 1) {
  xValues.push(x);
  yValues.push(x * slope);
}
```

教程說明：此例中的斜率是一個計算出來的平均值，而截距 = 0。

方法三：線性迴歸函數（最小二乘法）

這個模型用**最小二乘法**公式同時計算斜率與截距（S16「Using a Linear Regression Function」）：

S16「Using a Linear Regression Function」

```
// Calculate Sums
let xSum=0, ySum=0 , xxSum=0, xySum=0;
```

```
let count = xArray.length;
for (let i = 0, len = count; i < count; i++) {
  xSum += xArray[i];
  ySum += yArray[i];
  xxSum += xArray[i] * xArray[i];
  xySum += xArray[i] * yArray[i];
}

// Calculate slope and intercept
let slope = (count * xySum - xSum * ySum) / (count * xxSum - xSum * xSum);
let intercept = (ySum / count) - (slope * xSum) / count;

// Generate values
const xValues = [];
const yValues = [];
for (let x = 50; x <= 150; x += 1) {
  xValues.push(x);
  yValues.push(x * slope + intercept);
}
```

$$\text{斜率 } a = (n \cdot \Sigma xy - \Sigma x \cdot \Sigma y) \div (n \cdot \Sigma x^2 - (\Sigma x)^2)；\text{截距 } b = \bar{y} - a \cdot \bar{x}$$

【補充說明（非來源內容）】 以教程的 11 對數據代入：方法二的平均斜率 = 114 ÷ 1100 ≈ 0.1036（截距 0）；方法三的最小二乘斜率 ≈ **0.0791**、截距 ≈ **2.4545**，即預測直線約為 y = 0.0791x + 2.4545。例如面積 100 平方米的預測價格約為 10.36（百萬）。這些數值由講義編者計算，可用來核對自己的程式輸出。

5.6.2 多項式迴歸（Polynomial Regression）

若散點不適合線性迴歸（一條穿過各點的直線），數據可能適合**多項式迴歸**。多項式迴歸與線性迴歸一樣，利用 x 與 y 之間的關係找出穿過數據點的最佳畫線方法（S16「Polynomial Regression」；教程附有一幅曲線擬合示意圖）。

【學習提示】 S16 的三種方法與第 7 節的梯度下降可視為「同一個問題的四種解法」：手繪、平均斜率、代數公式（最小二乘法封閉解），以及以迭代方式逐步逼近最佳直線的機器學習方法。教程在 S12 明言：「這可以用數學公式解決，但機器學習演算法同樣能解決。」

6. 感知器（Perceptron）：原理、程式實作、訓練與測試

感知器是**人工神經元（Artificial Neuron）**，是最簡單的神經網絡；而神經網絡是機器學習的基本構件（S08 開首）。本節依照 S08（原理）→ S09（模式識別問題）→ S10（訓練）→ S11（測試）的順序，完整重現教程如何用 JavaScript 由零開始建造並訓練一個感知器。

6.1 Frank Rosenblatt 與感知器的起源

Frank Rosenblatt（1928–1971）是美國心理學家，在人工智能領域享負盛名。**1957 年**他在康奈爾航空實驗室（Cornell Aeronautical Laboratory）的 **IBM 704** 電腦上「發明」了感知器程式（S08「Frank Rosenblatt」）。

當時科學家已發現腦細胞（神經元）透過電訊號接收來自感官的輸入；神經元再以電訊號儲存資訊，並根據先前的輸入作出決定。Rosenblatt 的構想是：感知器可以模擬大腦的原理，具備學習與決策的能力（S08）。

6.2 感知器的設計

原始的感知器設計為接收**多個二進制輸入**，並產生**一個二進制輸出**（0 或 1）。構想是用不同的**權重（weights）**代表每個輸入的重要性，而各值的總和必須大於一個**閾值（threshold）**，才會作出「是／否」（true／false、0／1）的決定（S08「The Perceptron」）。

6.2.1 去不去音樂會？一個直觀例子

想像你腦中有一個感知器，要決定是否去音樂會：歌手好嗎？天氣好嗎？這些因素應有甚麼權重？（S08「Perceptron Example」）

準則	輸入	權重
歌手好（Artist is Good）	x_1 = 0 或 1	w_1 = 0.7
天氣好（Weather is Good）	x_2 = 0 或 1	w_2 = 0.6
朋友會來（Friend will Come）	x_3 = 0 或 1	w_3 = 0.5
供應食物（Food is Served）	x_4 = 0 或 1	w_4 = 0.3
供應酒精（Alcohol is Served）	x_5 = 0 或 1	w_5 = 0.4

6.2.2 感知器演算法（The Perceptron Algorithm）

Rosenblatt 建議的演算法有四步（S08「The Perceptron Algorithm」）：

1. **設定閾值**：Threshold = 1.5
2. **把每個輸入乘以其權重**：$x_1 \cdot w_1 = 1 \times 0.7 = 0.7$；$x_2 \cdot w_2 = 0 \times 0.6 = 0$；$x_3 \cdot w_3 = 1 \times 0.5 = 0.5$；$x_4 \cdot w_4 = 0 \times 0.3 = 0$；$x_5 \cdot w_5 = 1 \times 0.4 = 0.4$
3. **把所有結果相加**：0.7 + 0 + 0.5 + 0 + 0.4 = **1.6**（加權總和 The Weighted Sum）

4. 啟動輸出：若總和 > 1.5 則回傳 true（「是，我會去音樂會」）

教程補充：若天氣的權重對你是 0.6，對別人可能不同——權重越高表示天氣對那人越重要；若閾值對你是 1.5，對別人也可能不同——閾值越低表示那人越想去任何音樂會（S08「Note」）。

S08：以 JavaScript 實作感知器演算法

```
const threshold = 1.5;
const inputs = [1, 0, 1, 0, 1];
const weights = [0.7, 0.6, 0.5, 0.3, 0.4];

let sum = 0;
for (let i = 0; i < inputs.length; i++) {
  sum += inputs[i] * weights[i];
}

const activate = (sum > 1.5);
```

6.3 感知器的組成部分

感知器受生物神經元的功能啟發，在 AI 中扮演關鍵角色，是神經網絡的重要構件。教程把它拆解為以下部件（S08「Perceptron in AI」）：

部件	說明	例子中的值
1. 輸入（Inputs／nodes）	感知器接收一個或多個輸入，稱為節點（nodes）；節點同時具有**值**與**權重**。	5 個節點
2. 節點值（Node Values）	輸入節點的值是二進制的 1 或 0，可解讀為 true／false、yes／no。	1, 0, 1, 0, 1
3. 節點權重（Node Weights）	指派給每個輸入的值，顯示每個節點的強度；數值越高，該輸入對輸出的影響越大。	0.7, 0.6, 0.5, 0.3, 0.4
4. 加總（Summation）	計算輸入的加權總和：每個輸入乘以對應權重，再相加。	0.7×1 + 0.6×0 + 0.5×1 + 0.3×0 + 0.4×1 = 1.6
5. 閾值（Threshold）	感知器「觸發（fire）」（輸出 1）所需的值，否則保持不活躍（輸出 0）。	1.5
6. 啟動函數（Activation Function）	加總後套用的函數，目的是為輸出**引入非線性**；根據聚合後的輸入決定感知器應否觸發。	(sum > threshold) == (1.6 > 1.5)
7. 輸出（Output）	啟動函數的結果，代表感知器基於輸入與權重作出的決定或預測；把加權總和映射為二進制值（1／0，即 true／false）。	1，因為 (sum > threshold) == true

【來源問題／不一致之處】 S08 原文的小節編號次序為「1, 2, 3, 4, 6, 5」（「6. The Threshold」排在「5. The Activation Function」之前），且 threshold 多處拼作 “treshold”。講義已按邏輯次序重新編號為 1–7。

6.4 感知器如何學習；感知器的限制

感知器可以透過稱為**訓練（training）**的過程從例子中學習。訓練期間，感知器根據觀察到的誤差調整權重，通常使用**感知器學習規則（perceptron learning rule）**或**反向傳播（backpropagation）**演算法。學習過程向感知器呈現**已標記的例子**（已知期望輸出），感知器把自己的輸出與期望輸出比較並相應調整權重，目標是**把預測輸出與期望輸出之間的誤差減至最小**。這使感知器學到能對新的、未知的輸入作出準確預測的權重（S08「Perceptron Learning」）。

教程亦指出感知器的限制（S08「Note」「Neural Networks」）：

- 顯然單一個神經元不能作出決定，其他神經元必須提供更多輸入（歌手好嗎？天氣好嗎？……）；**多層感知器（Multi-Layer Perceptrons）**可用於更精密的決策。
- 感知器雖對人工神經網絡的發展影響深遠，但**只能學習線性可分（linearly separable）的模式**。
- 把多個感知器按層堆疊並加入非線性啟動函數，神經網絡便能克服這個限制，學習更複雜的模式。
- 感知器常作為更複雜神經網絡的構件，例如多層感知器（MLPs）或深度神經網絡（DNNs）；把多個感知器分層組合並連接成網絡，這些模型便能學習並表示數據中複雜的模式與關係，實現圖像識別、自然語言處理與決策等任務。

> **【補充說明（非來源內容）】** 「線性可分」的意思是：能用一條直線（二維）或一個平面／超平面（高維）把兩類數據完全分開。第 6.5 節的「直線上下分類」正是一個線性可分問題，所以單一感知器足以解決；經典的 XOR 問題則不是線性可分的，需要多層網絡——第 8.3 節 Brain.js 的第一個範例正是以多層網絡學習 XOR 樣式的對應。

6.5 模式識別：用感知器把點分成「線上」與「線下」

神經網絡用於人臉識別等應用，這些應用使用**模式識別（Pattern Recognition）**；這類分類可以用感知器完成——感知器能把數據分成兩部分，因此又稱為**線性二元分類器（Linear Binary Classifiers）**（S09 開首）。

問題設定（S09「Pattern Classification」）：想像一個散佈着 x–y 點的空間中有一條直線（線性圖），如何把直線上方與下方的點分類？**感知器可以在不知道直線公式的情況下，被訓練成能辨認線上方的點。**

6.5.1 程式步驟總覽

要編寫這個感知器，教程用一個簡單的 JavaScript 程式完成以下工作（S09「How to Program a Perceptron」）：

1. 建立一個簡單的繪圖器（plotter）
2. 建立 500 個隨機 x–y 點
3. 顯示這些點

4. 建立直線函數 f(x)
5. 顯示直線
6. 計算期望答案（desired answers）
7. 顯示期望答案

6.5.2 逐步程式碼

步驟 1：建立繪圖器（XYPlotter 物件的定義見第 11.1.2 節）

```
const plotter = new XYPlotter("myCanvas");
plotter.transformXY();
const xMax = plotter.xMax;
const yMax = plotter.yMax;
const xMin = plotter.xMin;
const yMin = plotter.yMin;
```

步驟 2–3：建立並顯示 500 個隨機點（x、y 均介乎 0 與最大值之間）

```
const numPoints = 500;
const xPoints = [];
const yPoints = [];
for (let i = 0; i < numPoints; i++) {
  xPoints[i] = Math.random() * xMax;
  yPoints[i] = Math.random() * yMax;
}
```

步驟 4–5：直線函數 y = 1.2x + 50

```
function f(x) {
  return x * 1.2 + 50;
}
```

步驟 6：計算期望答案——點在線上方則為 1，否則為 0，存入 desired[]

```
let desired = [];
for (let i = 0; i < numPoints; i++) {
  desired[i] = 0;
  if (yPoints[i] > f(xPoints[i])) {desired[i] = 1;}
}
```

步驟 7：顯示期望答案——desired[i] = 1 畫黑點，否則畫藍點

```
for (let i = 0; i < numPoints; i++) {
  let color = "blue";
  if (desired[i]) color = "black";
  plotter.plotPoint(xPoints[i], yPoints[i], color);
}
```

下一步（S09「How to Train a Perceptron」）：利用這些正確答案訓練感知器，使它能預測未知輸入的輸出值。

6.6 訓練感知器

訓練任務：想像一個散佈着 x–y 點的空間中有一條直線，訓練感知器把直線上方與下方的點分類。步驟是：建立感知器物件 → 建立訓練函數 → 以正確答案訓練感知器（S10 開首及「Training Task」；頁面附有「Click to Train Me」互動示範）。

6.6.1 建立感知器物件

建立一個感知器物件（名稱任意，例如 Perceptron），接受兩個參數：輸入數目（no）與學習率（learningRate，預設 0.00001）；然後為每個輸入建立介乎 −1 與 1 的隨機權重（S10「Create a Perceptron Object」）：

S10：感知器物件（注意迴圈用 <= no，因此權重數目比輸入多一個，留給偏置）

```
// Perceptron Object
function Perceptron(no, learningRate = 0.00001) {

// Set Initial Values
this.learnc = learningRate;
this.bias = 1;

// Compute Random Weights
this.weights = [];
for (let i = 0; i <= no; i++) {
  this.weights[i] = Math.random() * 2 - 1;
}

// End Perceptron Object
}
```

概念	說明（S10）
隨機權重（Random Weights）	感知器以每個輸入一個隨機權重開始。
學習率（Learning Rate）	訓練期間每出現一個錯誤，權重會被調整一個很小的比例；這個小比例就是「感知器的學習率」，在物件中稱為 learnc。
偏置（Bias）	有時當兩個輸入都是零，感知器可能產生錯誤輸出；為避免這情況，給感知器一個額外的、值為 1 的輸入，稱為偏置。

6.6.2 加入啟動函數

回顧感知器演算法：每個輸入乘以權重 → 把結果相加 → 計算結果。啟動函數在總和大於 0 時輸出 1，否則輸出 0（S10「Add an Activate Function」）：

S10：啟動函數

```
this.activate = function(inputs) {
  let sum = 0;
  for (let i = 0; i < inputs.length; i++) {
    sum += inputs[i] * this.weights[i];
  }
  if (sum > 0) {return 1} else {return 0}
}
```

【補充說明（非來源內容）】 與 6.2 節的例子比較，這裏的閾值變成了 0。這是因為偏置輸入（值 1）配上可學習的權重，等同於一個可調整的閾值：sum > 0 等價於「其他輸入的加權總和 > −(偏置權重)」。這是感知器實作中的標準做法。

6.6.3 建立訓練函數

訓練函數根據啟動函數猜測結果；每次猜錯，感知器都應調整權重；經過多次猜測與調整後，權重便會正確（S10「Create a Training Function」）：

S10：訓練函數

```
this.train = function(inputs, desired) {
  inputs.push(this.bias);
  let guess = this.activate(inputs);
  let error = desired - guess;
  if (error != 0) {
    for (let i = 0; i < inputs.length; i++) {
      this.weights[i] += this.learnc * error * inputs[i];
    }
  }
}
```

逐行理解：

1. `inputs.push(this.bias)`：把偏置（1）加到輸入陣列末端。
2. `guess = this.activate(inputs)`：用目前權重作出猜測（0 或 1）。
3. `error = desired - guess`：誤差只可能是 0（猜對）、+1（應為 1 卻猜 0）或 −1（應為 0 卻猜 1）。
4. 若 error ≠ 0，每個權重加上 `learnc × error × inputs[i]`——猜低了就把權重往上推，猜高了就往下拉，推動幅度與輸入值成正比。

反向傳播（Backpropagation）：每次猜測後，感知器計算猜測錯了多少；若猜錯，便調整偏置與權重，令下次猜測「稍微更正確」。教程稱這種學習為反向傳播；經過（數千次）嘗試後，感知器會變得相當擅長猜測（S10「Backpropagation」）。

【補充說明（非來源內容）】 嚴格而言，上述單層感知器所用的是「感知器學習規則」；「反向傳播」

> 一詞在現代用法中通常指多層網絡中以鏈式法則把誤差梯度由輸出層逐層傳回的演算法。教程在此把兩者等同看待，是簡化的說法。

6.6.4 建立自己的程式庫並使用

教程把感知器物件（初始值、隨機權重、activate、train）合併成一個完整程式庫，儲存為 myperceptron.js 後以 `<script src="myperceptron.js"></script>` 引入（S10「Create Your Own Library」）。使用程式庫的完整流程如下（S10「Use Your Library」）：

S10：使用感知器程式庫（2 個輸入、10,001 輪 × 500 點的訓練）

```
// Initiate Values
const numPoints = 500;
const learningRate = 0.00001;

// Create a Plotter
const plotter = new XYPlotter("myCanvas");
plotter.transformXY();
const xMax = plotter.xMax;
const yMax = plotter.yMax;
const xMin = plotter.xMin;
const yMin = plotter.yMin;

// Create Random XY Points
const xPoints = [];
const yPoints = [];
for (let i = 0; i < numPoints; i++) {
  xPoints[i] = Math.random() * xMax;
  yPoints[i] = Math.random() * yMax;
}

// Line Function
function f(x) {
  return x * 1.2 + 50;
}

//Plot the Line
plotter.plotLine(xMin, f(xMin), xMax, f(xMax), "black");

// Compute Desired Answers
const desired = [];
for (let i = 0; i < numPoints; i++) {
  desired[i] = 0;
  if (yPoints[i] > f(xPoints[i])) {desired[i] = 1}
}

// Create a Perceptron
const ptron = new Perceptron(2, learningRate);

// Train the Perceptron
```

```
for (let j = 0; j <= 10000; j++) {
  for (let i = 0; i < numPoints; i++) {
    ptron.train([xPoints[i], yPoints[i]], desired[i]);
  }
}

// Display the Result
for (let i = 0; i < numPoints; i++) {
  const x = xPoints[i];
  const y = yPoints[i];
  let guess = ptron.activate([x, y, ptron.bias]);
  let color = "black";
  if (guess == 0) color = "blue";
  plotter.plotPoint(x, y, color);
}
```

> **【學習提示】** 訓練迴圈 `for (let j = 0; j <= 10000; j++)` 會把全部 500 個點重複訓練 10,001 次，共約 500 萬次 train 呼叫。顯示結果時，必須手動把 `ptron.bias` 加到輸入陣列，因為 activate 本身不會加入偏置（只有 train 會 push）。

6.7 測試與調校感知器

感知器必須被**測試與評估**，而且必須以**真實值**測試（S11 開首）。方法是產生新的、未知的點，檢查感知器能否猜對（S11「Test Your Library」），並加入計數器統計錯誤（S11「Count the Errors」）：

S11：以 500 個新點測試並計算錯誤數

```
// Test Against Unknown Data
const counter = 500;
let errors = 0;
for (let i = 0; i < counter; i++) {
  let x = Math.random() * xMax;
  let y = Math.random() * yMax;
  let guess = ptron.activate([x, y, ptron.bias]);
  let color = "black";
  if (guess == 0) color = "blue";
  plotter.plotPoint(x, y, color);
  if ((y > f(x) && guess == 0) || (y < f(x) && guess == 1)) {errors++}
}
```

如何調校感知器？教程建議（S11「Tune the Perceptron」）：

- 調整學習率（learning rate）
- 增加訓練數據的數量
- 增加訓練迭代的次數

> **【學習提示】** 用**未見過的新數據**測試而非用訓練數據本身，是機器學習評估的基本原則——對應第 10 節 TensorFlow 範例中「It Evaluates the Model」一步。

7. 學習即迴圈：成本函數與梯度下降（Gradient Descent）

S12「Machine Learning」頁面以一個線性迴歸的訓練器（Trainer）物件示範機器學習最核心的機制：反覆迭代、量度誤差、朝誤差減少的方向調整參數。

7.1 學習就是迴圈（Learning is Looping）

機器學習模型是透過**對數據多次迴圈**來訓練的。每次迭代都會調整**權重值**。當迭代無法再**降低成本（Cost）**時，訓練便完成（S12「Learning is Looping」）。頁面附有一個互動示範：「訓練我找出最佳擬合線」，可選擇訓練 100、200、300 或 500 次，觀察直線逐步貼近散點。

7.2 梯度下降的概念

梯度下降是解決 AI 問題的流行演算法。教程用一個簡單的線性迴歸模型示範（S12「Gradient Descent」）：

- 線性迴歸的目標是把一條線性圖**擬合**到一組 (x, y) 點。這可以用數學公式解決（見第 5.6 節），但機器學習演算法同樣能解決。
- 示範由一幅散點圖與一個線性模型 **y = wx + b** 開始。
- 然後訓練模型找出一條貼合散點的直線——方法是**改變直線的權重（斜率）與偏置（截距）**。

7.3 訓練器物件（Trainer Object）

建立一個 Trainer 物件，可接受任意數量的 (x, y) 值（兩個陣列 xArr、yArr）。權重設為 0，偏置設為 1；必須設定學習常數（learnc）並定義成本變數（S12「A Trainer Object」）：

S12：Trainer 物件的初始值

```
function Trainer(xArray, yArray) {
  this.xArr = xArray;
  this.yArr = yArray;
  this.points = this.xArr.length;
  this.learnc = 0.00001;
  this.weight = 0;
  this.bias = 1;
  this.cost;
```

7.4 成本函數（Cost Function）

解決迴歸問題的標準方法是用一個「成本函數」量度解答有多好。該函數使用模型（y = wx + b）的權重與偏置，根據直線與散點的貼合程度回傳一個誤差。計算方法是遍歷所有 (x, y) 點，把每點的 y 值與直線之間的距離**平方**後相加。平方距離是最常規的做法，一來確保數值為正，二來使誤差函數**可微分（differentiable）**（S12「Cost Function」）。成本函數的另一名稱是**誤差函數（Error Function）**。

S12：成本函數（均方誤差）

```
this.costError = function() {
  total = 0;
  for (let i = 0; i < this.points; i++) {
    total += (this.yArr[i] - (this.weight * this.xArr[i] + this.bias)) **2;
  }
  return total / this.points;
}
```

函數實際使用的公式（教程以圖片呈現）是：

$$E = (1/N) \cdot \Sigma_{i=1}^{N} (y_i - (m \cdot x_i + b))^2$$

符號	意義（S12）
E	誤差（成本）
N	觀察（點）的總數
y	每個觀察的值（標籤）
x	每個觀察的值（特徵）
m	斜率（權重）
b	截距（偏置）
mx + b	預測值
(1/N) · Σ	平方值的平均（教程原文寫作 “the squared mean value”）

【補充說明（非來源內容）】 這個公式就是**均方誤差（Mean Squared Error, MSE）**，也是第 9–10 節 TensorFlow.js 範例中 `loss: 'meanSquaredError'` 所指的損失函數。「成本函數／誤差函數／損失函數」在本教程中可視為同義。

7.5 訓練函數與更新權重函數

現在執行梯度下降：梯度下降演算法應「沿着成本函數走向最佳直線」，每次迭代都應把 m 與 b 更新為成本（誤差）更低的直線。為此加入一個對所有數據迴圈多次的 train 函數（S12「The Train Function」）：

S12：訓練函數——迭代 iter 次，最後記錄成本

```
this.train = function(iter) {
  for (let i = 0; i < iter; i++) {
    this.updateWeights();
  }
  this.cost = this.costError();
}
```

train 函數應在每次迭代中更新權重與偏置；移動的方向以**兩個偏導數（partial derivatives）**計算（S12「An Update Weights Function」）：

S12：更新權重函數

```
this.updateWeights = function() {
  let wx;
  let w_deriv = 0;
  let b_deriv = 0;
  for (let i = 0; i < this.points; i++) {
    wx = this.yArr[i] - (this.weight * this.xArr[i] + this.bias);
    w_deriv += -2 * wx * this.xArr[i];
    b_deriv += -2 * wx;
  }
  this.weight -= (w_deriv / this.points) * this.learnc;
  this.bias -= (b_deriv / this.points) * this.learnc;
}
```

逐步解讀這個函數：

1. 對每個點計算殘差 `wx = y - (w * x + b)`（實際值減預測值）。
2. 累加對權重的偏導數貢獻 $-2 \cdot wx \cdot x$，以及對偏置的偏導數貢獻 $-2 \cdot wx$。
3. 把累加值除以點數得到平均梯度，再乘以學習率 learnc。
4. 用「減法」更新：`weight -= ...`、`bias -= ...`，即朝梯度的**反方向**（成本下降的方向）移動一小步。

> **【補充說明（非來源內容）】** **為何是 $-2 \cdot wx \cdot x$ 與 $-2 \cdot wx$？**（補充推導）成本 $E = (1/N)\Sigma(y_i - (w \cdot x_i + b))^2$。對 w 求偏導：$\partial E/\partial w = (1/N)\Sigma\, 2(y_i - (w \cdot x_i + b)) \cdot (-x_i) = (1/N)\Sigma(-2 \cdot wx_i \cdot x_i)$；對 b 求偏導：$\partial E/\partial b = (1/N)\Sigma(-2 \cdot wx_i)$。程式中的 w_deriv／points 與 b_deriv／points 正是這兩個偏導數。沿負梯度方向更新參數，便是「梯度下降」名稱的由來。

> **【補充說明（非來源內容）】** 這裏的 Trainer 每次迭代都用**全部**數據點計算梯度，屬「批量梯度下降（batch gradient descent）」；第 9–10 節 TensorFlow.js 使用的 `sgd`（隨機梯度下降）則以小批次（mini-batch）更新，原理相同但每步只看部分數據。

7.6 建立自己的程式庫

教程把以上三個函數合併為完整的 Trainer 程式庫（S12「Create Your Own Library」），儲存為 myailib.js 後以 `<script src="myailib.js"></script>` 引入：

S12：完整的 Trainer 程式庫（myailib.js）

```
function Trainer(xArray, yArray) {
  this.xArr = xArray;
  this.yArr = yArray;
  this.points = this.xArr.length;
  this.learnc = 0.00001;
```

```
  this.weight = 0;
  this.bias = 1;
  this.cost;

// Cost Function
this.costError = function() {
  total = 0;
  for (let i = 0; i < this.points; i++) {
    total += (this.yArr[i] - (this.weight * this.xArr[i] + this.bias)) **2;
  }
  return total / this.points;
}

// Train Function
this.train = function(iter) {
  for (let i = 0; i < iter; i++) {
    this.updateWeights();
  }
  this.cost = this.costError();
}

// Update Weights Function
this.updateWeights = function() {
  let wx;
  let w_deriv = 0;
  let b_deriv = 0;
  for (let i = 0; i < this.points; i++) {
    wx = this.yArr[i] - (this.weight * this.xArr[i] + this.bias);
    w_deriv += -2 * wx * this.xArr[i];
    b_deriv += -2 * wx;
  }
  this.weight -= (w_deriv / this.points) * this.learnc;
  this.bias -= (b_deriv / this.points) * this.learnc;
}

} // End Trainer Object
```

【補充說明（非來源內容）】 程式中 `total = 0;` 沒有以 let／const 宣告，會建立全域變數（在嚴格模式下會報錯）；`this.cost;` 一行只是「宣告意圖」，實際上不會建立屬性。這些是原始程式碼的小瑕疵，不影響理解。另外，學習率 0.00001 非常小，是因為 x 值（面積 50–150）與 y 值（價格 7–15）未經正規化；若把數據正規化到 0–1（見第 10.1 節），便可使用較大的學習率。

7.7 比較：感知器訓練 vs 梯度下降訓練器

	感知器（第 6 節）	**線性迴歸訓練器（第 7 節）**
任務類型	分類（點在線上／線下，輸出 0／1）	迴歸（預測連續的 y 值）

	感知器（第 6 節）	**線性迴歸訓練器（第 7 節）**
模型	sum = $\Sigma\, w_i x_i$（含偏置輸入 1）→ 啟動函數	$y = w \cdot x + b$
誤差量度	error = desired − guess（取值 −1、0、1）	均方誤差 $E = (1/N)\Sigma(y - (wx + b))^2$
更新規則	w_i += learnc × error × x_i（只在猜錯時）	w −= learnc × $\partial E/\partial w$；b −= learnc × $\partial E/\partial b$（每次迭代）
學習率	0.00001（預設）	0.00001
停止條件	固定迭代次數（10,001 輪）	固定迭代次數；教程原則：「迭代無法再降低成本」時完成

8. 機器學習的程式語言與 JavaScript 生態

8.1 機器學習常用的程式語言

教程列出涉及機器學習與人工智能的程式語言：LISP、R、Python、C++、Java、JavaScript、SQL（S03 開首）。各語言的特點整理如下：

語言	教程要點（S03）
LISP	世界上第二古老的程式語言（1958 年），比 Fortran（1957 年）年輕一年。「Artificial Intelligence」一詞由發明 LISP 的 John McCarthy 創造。LISP 建基於**遞歸函數（Recursive Functions，自我修改的函數）**理論，非常適合「自我學習」是重要部分的機器學習程式。
R	用於圖形與統計計算的程式語言，由 R Foundation for Statistical Computing 支持。附有大量統計與圖形技術：線性建模、非線性建模、統計檢定、時間序列分析、分類、聚類。
Python	通用編程語言，可用於各類程式與軟件開發；通常用於伺服器開發（如為網頁伺服器建立網頁應用）及數據科學。優點是附有非常合適的程式庫：**NumPy**（陣列）、**SciPy**（統計科學）、**Matplotlib**（繪圖）、**NLTK**（自然語言工具包）、**TensorFlow**（機器學習）。（教程附有一幅來自 Octoverse 的 2020 年語言流行度圖。）
C++	被稱為「世界上最快的程式語言」。因為速度快，是編寫電腦遊戲的首選語言；執行更快、回應時間更短，應用於搜尋引擎與遊戲開發。Google 在 SEO（搜尋引擎優化）的 AI 與 ML 程式中使用 C++。**SHARK** 是超快的 C++ 程式庫，支援監督式學習演算法、線性迴歸、神經網絡與聚類；**MLPACK** 也是超快的 C++ 機器學習程式庫。
Java	另一種通用編程語言，可用於各類軟件開發；在機器學習中主要用於建立演算法與神經網絡。
JavaScript	見 8.2 節。
SQL	SQL（Structured Query Language）是管理數據最流行的語言。認識 SQL 數據庫、表格與查詢有助數據科學家處理數據；SQL 對於在數據庫中儲存、操作與擷取數據非常方便。

8.2 為何用 JavaScript 做機器學習？

傳統上機器學習應用使用 R 或 Python，但教程認為 JavaScript 作為機器學習語言「前途光明」（S04 開首）：

- JavaScript 廣為人知，所有開發者都會用。
- 內置安全性：JavaScript 無法存取你的檔案。
- JavaScript 比 Python 快。
- JavaScript 可使用硬件加速。
- JavaScript 在瀏覽器中執行。

機器學習可能數學繁重，神經網絡的性質高度技術性，隨之而來的術語往往把人嚇跑；JavaScript 以易於理解的軟件簡化建立與訓練神經網絡的過程。有了新的機器學習程式庫，JavaScript 開發者可以把 ML 與 AI 加入網頁應用（S04「JavaScript is Good for Machine Learning」）。

「在瀏覽器中做機器學習」意味着：JavaScript 中的機器學習、為網頁而設的機器學習、人人可用的機器學習、更多平台上的機器學習（S04「JavaScript Machine Learning Libraries」）。優點包括：

- 易於使用，無需安裝任何東西。
- 強大的圖形：瀏覽器支援 WebGL。
- 更好的私隱：數據可以留在客戶端。
- 更多平台：JavaScript 可在流動裝置上執行。

> **【補充說明（非來源內容）】** 「JavaScript 比 Python 快」是 S04 的原話，屬概括性說法；實際效能取決於工作負載與所用的底層程式庫（例如 Python 的 NumPy／TensorFlow 以 C／C++／GPU 執行密集運算）。

8.2.1 JavaScript 機器學習與數學程式庫一覽

程式庫／工具	教程描述（S04）
Brain.js	讓人容易理解神經網絡的 JavaScript 程式庫，因為它隱藏了數學的複雜性；簡單易用，不必深入認識神經網絡也能使用；提供多種神經網絡實作，因為不同的神經網絡擅長不同的事。（詳見 8.3 節）
ml5.js	致力讓機器學習更易被更廣泛的受眾接觸；ml5 團隊以更友善的方式包裝機器學習功能。教程示範只用三行程式便能分類圖像（見下方程式碼）。
TensorFlow Playground	以 d3.js 編寫的網頁應用，可以在不涉及數學的情況下學習神經網絡——在自己的瀏覽器中建立神經網絡並觀察結果。
TensorFlow.js	先前稱為 Tf.js 與 Deeplearn.js。（詳見第 9 節）
Math.js	為 JavaScript 與 Node.js 而設的廣泛數學程式庫，強大而易用；附有大量內置函數、靈活的運算式解析器，並支援數字、大數、複數、分數、單位、陣列與矩陣等多種數據類型。
繪圖程式庫	HTML Canvas、Plotly.js、Chart.js、Google Chart、D3.js（詳見第 11 節）。頁面另有「Plotting Equations」與「Plotting Values」兩個互動工具，可輸入方程或 x／y 值並繪成散點或直線。
WebGL API	在任何瀏覽器中渲染 2D 與 3D 圖形的 JavaScript API；可在任何 PC 的集成或獨立顯示卡上執行；把 3D 圖形帶到網頁瀏覽器。主要瀏覽器廠商 Apple（Safari）、Google（Chrome）、Microsoft（Edge）與 Mozilla（Firefox）都是 WebGL 工作小組成員。

S04：ml5.js 三行圖像分類（教程建議把 pic1.jpg 換成 pic2.jpg、pic3.jpg、pic4.jpg 試驗）

```
<img id="myImage" src="pic1.jpg" width="100%">

<script>
const classifier = ml5.imageClassifier('MobileNet');
classifier.classify(document.getElementById("myImage"), gotResult);
function gotResult(error, results) { ... }
</script>
```

8.3 Brain.js 實作

Brain.js 隱藏了數學的複雜性，讓神經網絡易於理解（S18 開首）。

8.3.1 第一個神經網絡：學習 [0,0]→zero、[0,1]→one、[1,0]→one、[1,1]→zero

S18「Building a Neural Network」

```
// Create a Neural Network
const network = new brain.NeuralNetwork();

// Train the Network with 4 input objects
network.train([
  {input:[0,0], output:{zero:1}},
  {input:[0,1], output:{one:1}},
  {input:[1,0], output:{one:1}},
  {input:[1,1], output:{zero:1}},
]);

// What is the expected output of [1,0]?
result = network.run([1,0]);

// Display the probability for "zero" and "one"
... result["one"] + " " + result["zero"];
```

範例說明（S18「Example Explained」）：

- 以 `new brain.NeuralNetwork()` 建立神經網絡。
- 以 `network.train([examples])` 訓練網絡；例子代表 4 個輸入值及其對應的輸出值。
- 以 `network.run([1,0])` 詢問「[1,0] 最可能的輸出是甚麼？」
- 網絡的回答：**one：93%**（接近 1）、**zero：6%**（接近 0）。

> **【來源問題／不一致之處】** S18 原始程式碼中第三、四個訓練物件缺少右大括號（`{input:[1,0], output:{one:1},`），講義已補正。這四個例子正是 XOR（互斥或）的真值表——單層感知器無法學習，但 Brain.js 預設的多層網絡可以。

8.3.2 預測顏色的深淺（對比度）

CSS 可用 RGB 設定顏色（S18「How to Predict a Contrast」）：

顏色	RGB
黑色（Black）	RGB(0,0,0)
黃色（Yellow）	RGB(255,255,0)
紅色（Red）	RGB(255,0,0)
白色（White）	RGB(255,255,255)
淺灰（Light Gray）	RGB(192,192,192)

顏色	RGB
深灰（Dark Gray）	RGB(65,65,65)

S18：預測深藍色 (0,0,128) 是深色還是淺色

```
// Create a Neural Network
const net = new brain.NeuralNetwork();

// Train the Network with 4 input objects
net.train([
// White RGB(255, 255, 255)
{input:[255/255, 255/255, 255/255], output:{light:1}},
// Light grey (192,192,192)
{input:[192/255, 192/255, 192/255], output:{light:1}},
// Darkgrey (64, 64, 64)
{ input:[65/255, 65/255, 65/255], output:{dark:1}},
// Black (0, 0, 0)
{ input:[0, 0, 0], output:{dark:1}},
]);

// What is the expected output of Dark Blue (0, 0, 128)?
let result = net.run([0, 0, 128/255]);

// Display the probability of "dark" and "light"
... result["dark"] + " " + result["light"];
```

網絡的回答：**Dark：95%、Light：4%**。教程鼓勵讀者修改範例，測試黃色或紅色的可能輸出（S18「Example Explained」）。

> **【來源問題／不一致之處】** 把 RGB 值除以 255 是把輸入正規化到 0–1 範圍，與第 10 節的 min-max 正規化同一道理。另外，程式註釋寫 “Darkgrey (64, 64, 64)” 而程式碼與上表用的是 65，屬來源的小不一致。

8.4 機器學習應用實例

S05「Machine Learning Examples」頁面介紹數個應用例子：

8.4.1 圖像分類示範

頁面頂部有一個互動的圖像分類器（按鈕「Click to Classify Picture」與「Click to Change Picture」），即 8.2.1 節 ml5.js 範例的實際應用（S05「Image Classification Example」）。

8.4.2 人工音樂智能：David Cope 與 EMI

演算法能否作出比人類更好的音樂？David Cope 是加州聖塔克魯茲大學（University of Santa Cruz）前音樂教授，三十多年來一直開發 **Emmy／EMI（Experimental Musical Intelligence）**——一個以著名作曲家風格作曲的演算法（S05「Artificial Music Intelligence」）。

在奧勒岡大學教授 Douglas Hofstadter 進行的一次測試中，一位鋼琴家演奏三首巴赫風格的樂曲：一首由巴赫所寫、一首由 Steve Larson 所寫、一首由 EMI 所寫。當聽眾判定 Larson 的作品是 EMI 所寫時，Larson 博士感到受傷；當聽眾判定 EMI 的作品是真正的巴赫時，他才好過一些（S05「Bach, Larson, or EMI?」；來源：New York Times。頁面另附「David Cope Emmy Vivaldi」的影片／圖片）。

8.4.3 Project Baseline

Project Baseline 是一項倡議，讓每個人都能輕易為「人類健康地圖」作出貢獻並參與臨床研究；研究員、臨床醫生、工程師、設計師、倡議者與義工可以合作建立下一代醫療工具與服務（S05「Project Baseline」）。

8.4.4 數據科學家

數據科學家可以是多個學科的專家：應用數學、計算統計、電腦科學、機器學習、深度學習；他們亦具備豐富的大數據經驗：商業智能、數據庫設計、數據倉庫設計、數據挖掘、SQL 查詢、SQL 報表（S05「Data Scientists」）。

8.4.5 人工健康智能與 FDA 聲明

新冠疫情推動了優化醫療保健的需求。機器學習是一項新技術，能帶來更好的藥物發現、更短的開發時間與更低的藥物成本；機器學習讓醫療保健能利用「大數據」作出更好的醫療或臨床決定（S05「Artificial Health Intelligence」）。

> *Artificial intelligence and machine learning have the potential to fundamentally transform the delivery of health care. As technology and science advance, we can expect to see earlier disease detection, more accurate diagnosis, more targeted therapies and significant improvements in personalized medicine.*
> *（人工智能與機器學習有潛力從根本上改變醫療服務的提供方式。隨着科技與科學進步，我們可以預期更早的疾病偵測、更準確的診斷、更有針對性的治療，以及個人化醫療的顯著改善。）*
>
> — FDA 局長 Scott Gottlieb, M.D.，就 AI 醫療器械新審查框架的聲明（引自 S05）

9. TensorFlow.js：張量、運算、模型與視覺化

9.1 甚麼是 TensorFlow.js？

TensorFlow 是流行的 JavaScript 機器學習程式庫，讓我們在瀏覽器中訓練與部署機器學習，並把機器學習功能加入任何網頁應用（S19「What is TensorFlow.js?」）。使用方法是在 HTML 檔案加入 script 標籤（S19「Using TensorFlow」）：

S19：引入 TensorFlow.js

```
<!-- 指定版本 -->
<script src="https://cdn.jsdelivr.net/npm/@tensorflow/tfjs@3.6.0/dist/tf.min.js"></script>

<!-- 永遠使用最新版本：去掉版本號 -->
<script src="https://cdn.jsdelivr.net/npm/@tensorflow/tfjs"></script>
```

歷史背景（S19）：TensorFlow 由 Google Brain 團隊為 Google 內部使用而開發，**2015 年**以開放軟件發佈；教程稱 **2019 年 1 月** Google 開發者發佈了 TensorFlow 的 JavaScript 實作 TensorFlow.js，其設計目標是提供與原本以 Python 編寫的 TensorFlow 程式庫相同的功能。

> **【來源問題／不一致之處】** 其他來源對 TensorFlow.js 的日期有不同記載：Google 於 **2018 年 3 月**在 TensorFlow Dev Summit 宣佈 TensorFlow.js（由 deeplearn.js 團隊併入而來），而 **1.0 版**於 **2019 年 3 月**發佈（見 Wikipedia「TensorFlow」條目及相關技術文章，第 14 節補充來源）。S19 所述「2019 年 1 月」與這些來源不符，讀者引用時宜留意。S04 亦提到 TensorFlow.js 先前稱為 Tf.js 與 Deeplearn.js。

9.2 張量（Tensors）

TensorFlow.js 是用以**定義與操作張量**的 JavaScript 程式庫；其主要數據類型是張量。張量與多維陣列大致相同，包含一個或多個維度的值（S19「Tensors」）。張量有三個主要屬性：

屬性	說明
dtype	數據類型
rank	維度的數目
shape	每個維度的大小

術語提示（S19）：機器學習中「維度（dimension）」有時與「秩（rank）」交替使用——[10, 5] 是一個 2 維張量，也就是秩為 2 的張量；而「維數（dimensionality）」可以指某一個維度的大小——在 2 維張量 [10, 5] 中，第一個維度的維數是 10。

9.2.1 建立張量

張量可用 `tf.tensor()` 由任何 N 維陣列建立（S19「Creating a Tensor」）：

S19：由巢狀陣列建立張量

```
const tensorA = tf.tensor([[1, 2, 3, 4]]);           // 例 1：1 × 4
const tensorB = tf.tensor([[1, 2], [3, 4]]);         // 例 2：2 × 2
const tensorC = tf.tensor([[1, 2], [3, 4], [5, 6]]); // 例 3：3 × 2
```

也可以由一個一維陣列加上 **shape** 參數建立（S19「Tensor Shape」）：

S19：以 shape 參數建立張量

```
const myArr = [1, 2, 3, 4];
const shape = [2, 2];
const tensorA = tf.tensor(myArr, shape);

// 等價寫法
const tensorB = tf.tensor([1, 2, 3, 4], [2, 2]);

// 已是 2 維的陣列也可再指定 shape
const tensorC = tf.tensor([[1, 2], [3, 4]], [2, 2]);
```

9.2.2 讀取張量的值與屬性

方法／屬性	作用（S19「Retrieve Tensor Values」）	範例
`tensor.data()`	取得張量背後的數據（回傳 Promise，扁平化的值）	`tensorA.data().then(data => display(data));`
`tensor.array()`	取得張量背後的（巢狀）陣列（回傳 Promise）	`tensorA.array().then(array => display(array[0]));`（顯示第一列）；`array[1]` 顯示第二列
`tensor.rank`	取得秩	`tensorA.rank`（[2,2] 的張量為 2）
`tensor.shape`	取得形狀	`tensorA.shape`（例如 2,2）
`tensor.dtype`	取得數據類型	`tensorA.dtype`（預設 float32）

S19：以 data() 讀取數值（非同步）

```
const myArr = [[1, 2], [3, 4]];
const shape = [2, 2];
const tensorA = tf.tensor(myArr, shape);
tensorA.data().then(data => display(data));

function display(data) {
  document.getElementById("demo").innerHTML = data;
}
```

> **【補充說明（非來源內容）】** `data()` 與 `array()` 都是**非同步**操作，回傳 Promise，因此必須用 `.then()`（或 `await`）取得結果；第 10 節的範例則使用同步版本 `dataSync()`。

9.2.3 張量的數據類型

張量可以有以下數據類型：bool、int32、**float32（預設）**、complex64、string。建立張量時可以把數據類型作為第三個參數指定（S19「Tensor Data Types」）：

S19：指定 dtype

```
const tensorA = tf.tensor([1, 2, 3, 4], [2, 2], "int32");
```

9.3 張量運算（TensorFlow Operations）

S20 介紹六種基本運算：加、減、乘、除、平方、重塑（reshape）。

運算	方法	範例輸入	結果（S20）
加法	`tensorA.add(tensorB)`	A = [[1,2],[3,4],[5,6]]；B = [[1,−1],[2,−2],[3,−3]]	[[2,1],[5,2],[8,3]]
減法	`tensorA.sub(tensorB)`	同上	[[0,3],[1,6],[2,9]]
乘法（逐元素）	`tensorA.mul(tensorB)`	A = [1,2,3,4]；B = [4,4,2,2]	[4,8,6,8]
除法（逐元素）	`tensorA.div(tensorB)`	A = [2,4,6,8]；B = [1,2,2,2]	[2,2,3,4]
平方	`tensorA.square()`	A = [1,2,3,4]	[1,4,9,16]
重塑	`tensorA.reshape([4, 1])`	A = [[1,2],[3,4]]	[[1],[2],[3],[4]]

關於重塑（S20「Tensor Reshape」）：張量的元素數目是 shape 中各大小的乘積；由於不同的形狀可以有相同的元素數目，把張量重塑為同樣大小的其他形狀往往很有用。

> **【補充說明（非來源內容）】** `mul()` 與 `div()` 是**逐元素（element-wise）**運算，與第 3.6.6 節的矩陣乘法（點積）不同；TensorFlow.js 的矩陣乘法另有 `matMul()` 方法（教程未提及）。

9.4 TensorFlow 模型（Models）

S21 把 TensorFlow.js 定位為「在瀏覽器中訓練與部署機器學習模型的 JavaScript 程式庫」。**模型（Models）與層（Layers）**是機器學習的重要構件：針對不同的任務，必須把不同類型的層組合成一個可以用數據訓練、以預測未來值的模型。TensorFlow.js 支援不同類型的模型與層；**一個 TensorFlow 模型就是一個具有一層或多層的神經網絡**（S21「Tensorflow Models」）。

9.4.1 TensorFlow 項目的典型工作流程

1. 收集數據（Collecting Data）

2. 建立模型（Creating a Model）
3. 為模型加入層（Adding Layers to the Model）
4. 編譯模型（Compiling the Model）
5. 訓練模型（Training the Model）
6. 使用模型（Using the Model）

9.4.2 範例：學習直線 Y = 1.2X + 5

假設已知一個定義直線的函數 Y = 1.2X + 5，用 JavaScript 可直接算出任何 y：`y = 1.2 * x + 5;`。為了示範 TensorFlow.js，教程改為訓練一個模型根據 X 輸入預測 Y 值——**模型並不知道這個函數**（S21「Example」「Note」）：

S21：完整範例

```
// Create Training Data
const xs = tf.tensor([0, 1, 2, 3, 4]);
const ys = xs.mul(1.2).add(5);

// Define a Linear Regression Model
const model = tf.sequential();
model.add(tf.layers.dense({units:1, inputShape:[1]}));

// Specify Loss and Optimizer
model.compile({loss:'meanSquaredError', optimizer:'sgd'});

// Train the Model
model.fit(xs, ys, {epochs:500}).then(() => {myFunction()});

// Use the Model
function myFunction() {
  const xMax = 10;
  const xArr = [];
  const yArr = [];
  for (let x = 0; x <= xMax; x++) {
    let result = model.predict(tf.tensor([Number(x)]));
    result.data().then(y => {
      xArr.push(x);
      yArr.push(Number(y));
      if (x == xMax) {plot(xArr, yArr)};
    });
  }
}
```

教程逐段解釋如下（S21「Collecting Data」至「Using the Model」）：

步驟	程式碼	說明
收集數據	`const xs = tf.tensor([0, 1, 2, 3, 4]);` `const ys = xs.mul(1.2).add(5);`	建立含 5 個 x 值的張量 xs，以及 5 個正確答案 ys（xs 乘 1.2 再加 5）。

步驟	程式碼	說明
建立模型	`const model = tf.sequential();`	建立**序列模型（sequential model）**：一層的輸出是下一層的輸入。
加入層	`model.add(tf.layers.dense({units:1, inputShape:[1]}));`	加入一個**密集層（dense layer）**：只有一個單元（張量），形狀為 1（一維）。在密集層中，**每個節點都與前一層的每個節點相連**。
編譯模型	`model.compile({loss:'meanSquaredError', optimizer:'sgd'});`	以 meanSquaredError 為損失函數、sgd（隨機梯度下降 stochastic gradient descent）為優化器。
訓練模型	`model.fit(xs, ys, {epochs:500}).then(...)`	用 xs 與 ys 訓練 500 次（epochs）；完成後呼叫 myFunction()。
使用模型	`model.predict(tf.tensor([Number(x)]))`	訓練後可作多種用途：此例對 x = 0…10 預測 y 值，並呼叫函數把預測繪成圖（另一版本改為 display() 顯示數值）。

【來源問題／不一致之處】 S21 說此例「預測 10 個 y 值」，但迴圈 `for (let x = 0; x <= xMax; x++)` 實際產生 x = 0 至 10 共 **11** 個預測；屬原文的小不一致。

9.4.3 TensorFlow 優化器一覽

教程列出 TensorFlow 提供的優化器（S21「Tensorflow Optimizers」）：

優化器	說明（S21）
Adadelta	實作 Adadelta 演算法
Adagrad	實作 Adagrad 演算法
Adam	實作 Adam 演算法
Adamax	實作 Adamax 演算法
Ftrl	實作 FTRL 演算法
Nadam	實作 NAdam 演算法
Optimizer	Keras 優化器的基類
RMSprop	實作 RMSprop 演算法
SGD	隨機梯度下降優化器

9.5 TensorFlow.js Visor（tfjs-vis）

TensorFlow Visor 是視覺化機器學習的圖形工具，包含視覺化 TensorFlow 模型的函數；視覺化可以組織在 **Visor（模態瀏覽器視窗）**中；可與 d3、Chart.js、Plotly.js 等自訂工具一起使用；通常稱為 **tfjs-vis**（S22 開首）。引入方法：

S22：引入 tfjs-vis

```
<script src="https://cdn.jsdelivr.net/npm/@tensorflow/tfjs-vis"></script>
```

三種圖表的用法如下；每種都有兩個版本——第一個參數是頁面元素（surface），或是 `{name: ...}` 物件（此時圖表會顯示在 Visor 視窗中）（S22「Scatter Plots」「Bar Graphs」「Line Graphs」）：

S22：散點圖（兩個系列，各 100 個隨機點）

```
const surface = document.getElementById('demo');
const series = ['First', 'Second'];
const serie1 = [];
const serie2 = [];
for (let i = 0; i < 100; i++) {
  serie1[i] = {x:i, y:Math.random() * 100};
  serie2[i] = {x:i, y:Math.random() * 100};
}
const data = {values: [serie1, serie2], series}
tfvis.render.scatterplot(surface, data);
// Visor 版本：tfvis.render.scatterplot({name: "my Plots"}, data);
```

S22：棒形圖

```
const data = [
  {index: 0, value: 100},
  {index: 1, value: 200},
  {index: 2, value: 150},
  {index: 2, value: 250},
];
tfvis.render.barchart(surface, data);
// Visor 版本：tfvis.render.barchart({name:"my Graphs"}, data);
```

S22：線形圖

```
let values = [
  {x: 1, y: 20},
  {x: 2, y: 30},
  {x: 3, y: 5},
  {x: 4, y: 12}
];
tfvis.render.linechart(surface, {values});
// Visor 版本：tfvis.render.linechart({name: 'my Lines'}, {values});
```

> **【來源問題／不一致之處】** 棒形圖範例中最後兩個數據點的 index 都是 2（`{index: 2, value: 150}` 與 `{index: 2, value: 250}`），很可能應為 2 與 3；屬原文筆誤。

10. TensorFlow.js 完整範例：汽車油耗與樓價預測

教程以兩個完整範例（Example 1：S23–S26；Example 2：S27–S30）示範一個 TensorFlow.js 項目的全流程。兩個範例**使用相同的原始碼**，只是數據集不同（S28 開首）。每個範例的介紹頁（S23、S27）都內嵌一個即時執行的示範，顯示「Input Data」散點圖與「Reduce Loss」損失曲線，並說明模型執行四個重要步驟：

1. 收集數據（It Collects Data）
2. 準備數據（It Prepares Data）
3. 訓練模型（It Trains a Model）
4. 評估模型（It Evaluates the Model）

隨後的章節教讀者編寫一個相同的程式：擷取並清理數據、建立 TensorFlow 模型、訓練模型（S23、S27「Explained」）。圖形以 tfjs-vis 顯示——一個為配合 TensorFlow.js 而設的瀏覽器內視覺化小型程式庫（S23、S27「Graphics (tfjs-vis)」；功能描述與第 9.5 節相同）。

10.1 範例一：以馬力預測每加侖英里數（汽車數據）

10.1.1 數據收集

範例一的數據是一個汽車物件清單，數據集是儲存於 https://storage.googleapis.com/tfjs-tutorials/carsData.json 的 JSON 檔案（S24「TensorFlow Data Collection」）。每個物件的格式如下：

S24：汽車數據物件（節錄）

```
{
  "Name": "chevrolet chevelle malibu",
  "Miles_per_Gallon": 18,
  "Cylinders": 8,
  "Displacement": 307,
  "Horsepower": 130,
  "Weight_in_lbs": 3504,
  "Acceleration": 12,
  "Year": "1970-01-01",
  "Origin": "USA"
},
{
  "Name": "buick skylark 320",
  "Miles_per_Gallon": 15,
  "Cylinders": 8,
  "Displacement": 350,
  "Horsepower": 165,
  "Weight_in_lbs": 3693,
  "Acceleration": 11.5,
  "Year": "1970-01-01",
  "Origin": "USA"
},
```

10.1.2 清理數據：移除不需要的數據與錯誤

為機器學習準備數據時，永遠要做兩件事：**移除不需要的數據**；**清除數據中的錯誤**（S24「Cleaning Data」）。

- **移除數據**：聰明的做法是只抽取需要的數據——用 map 函數遍歷數據。下面的函數接收一個物件，只回傳由 Horsepower（馬力）與 Miles_per_Gallon（每加侖英里數）組成的 x 與 y（S24「Remove Data」）。
- **移除錯誤**：大多數數據集都含有某類錯誤；聰明的做法是用 filter 函數把錯誤過濾掉。下面的函數在 x 或 y 任一屬性為 null 時回傳 false（S24「Remove Errors」）。

S24：map 與 filter 所用的函數

```
function extractData(obj) {
  return {x:obj.Horsepower, y:obj.Miles_per_Gallon};
}

function removeErrors(obj) {
  return obj.x != null && obj.y != null;
}
```

10.1.3 擷取數據與繪圖

有了 map 與 filter 函數，便可以寫一個擷取數據的函數（S24「Fetching Data」），並用 tfjs-vis 繪製散點圖（S24「Plotting the Data」）：

S24：擷取（async/await + fetch）與繪圖

```
async function runTF() {
  const jsonData = await fetch("cardata.json");
  let values = await jsonData.json();
  values = values.map(extractData).filter(removeErrors);
}

function tfPlot(values, surface) {
  tfvis.render.scatterplot(surface,
    {values:values, series:['Original','Predicted']},
    {xLabel:'Horsepower', yLabel:'MPG'});
}
```

> **【補充說明（非來源內容）】** `fetch("cardata.json")` 讀取的是本地檔案名，與上文的雲端網址 carsData.json 不同；實際練習時可把 JSON 下載到自己的伺服器目錄，或直接使用完整網址。

10.1.4 洗牌（Shuffle）數據

訓練前永遠要先洗牌數據。模型訓練時，數據會分成小組（批次 batches），每個批次逐一餵給模型。洗牌很重要，可防止模型重複得到相同的數據；若重複使用相同數據，模型將無法概括（generalize）數據並給出正確輸出；洗牌讓每個批次的數據更多樣化（S25「Shuffle Data」）：

S25：洗牌

```
tf.util.shuffle(data);
```

10.1.5 轉換成張量

要使用 TensorFlow，輸入數據必須轉換成張量數據（S25「TensorFlow Tensors」）：

S25：建立 2 維張量（n 列 × 1 欄）

```
// Map x values to Tensor inputs
const inputs = values.map(obj => obj.x);
// Map y values to Tensor labels
const labels = values.map(obj => obj.y);

// Convert inputs and labels to 2d tensors
const inputTensor = tf.tensor2d(inputs, [inputs.length, 1]);
const labelTensor = tf.tensor2d(labels, [labels.length, 1]);
```

10.1.6 數據正規化（Normalization）

數據在用於神經網絡之前應先正規化。對數值數據而言，用 min-max 把範圍壓到 0–1 通常最好（S25「Data Normalization」）：

S25：min-max 正規化

```
const inputMin = inputTensor.min();
const inputMax = inputTensor.max();
const labelMin = labelTensor.min();
const labelMax = labelTensor.max();

const nmInputs = inputTensor.sub(inputMin).div(inputMax.sub(inputMin));
const nmLabels = labelTensor.sub(labelMin).div(labelMax.sub(labelMin));
```

$$\text{正規化值} = (x - \min) \div (\max - \min) \quad \Rightarrow \quad \text{範圍 0 至 1}$$

10.1.7 定義模型

機器學習模型是一個由輸入產生輸出的演算法。此範例只用 3 行定義一個 ML 模型（S25「Tensorflow Model」）：

S25：模型定義

```
const model = tf.sequential();
model.add(tf.layers.dense({inputShape: [1], units: 1, useBias: true}));
```

```
model.add(tf.layers.dense({units: 1, useBias: true}));
```

概念	說明（S25）
序列模型（Sequential ML Model）	`tf.sequential()` 建立序列模型：輸入直接流向輸出；其他模型可以有多個輸入與多個輸出。序列是最簡單的 ML 模型，讓你逐層建立模型，權重對應到下一層。
層（Layers）	`model.add()` 用來加入兩個層。`tf.layers.dense` 是在大多數情況下都適用的層類型：**把輸入乘以一個權重矩陣，再加上一個數字（偏置）**。
形狀與單元（Shapes and Units）	`inputShape: [1]`：因為只有 1 個輸入（x = 馬力）。`units: 1`：定義權重矩陣的大小——每個輸入（x 值）一個權重。

> **【補充說明（非來源內容）】** 教程說「用 3 行定義模型」，而第 9.4.2 節的簡單範例只有一個 dense 層。這裏疊了兩個各有一個單元的 dense 層（皆無啟動函數），數學上仍等價於一條直線 y = w・x + b 的線性模型。

10.1.8 編譯模型

以指定的優化器與損失函數編譯模型：優化器設為 **sgd**，簡單易用而且相當有效；**meanSquaredError** 是用來比較模型預測與真實值的函數（S25「Compiling a Model」）：

S25：編譯

```
model.compile({loss: 'meanSquaredError', optimizer:'sgd'});
```

10.1.9 訓練模型

S26：訓練函數（批次 25，50 個週期）

```
async function trainModel(model, inputs, labels, surface) {
  const batchSize = 25;
  const epochs = 50;
  const callbacks = tfvis.show.fitCallbacks(surface, ['loss'], {callbacks:['onEpochEnd']})
  return await model.fit(inputs, labels,
    {batchSize, epochs, shuffle:true, callbacks:callbacks}
  );
}
```

- **epochs** 定義模型要做多少次迭代（迴圈）。
- **model.fit** 是執行迴圈的函數。
- **callbacks** 定義模型想重繪圖形時要呼叫的回呼函數——此處用 tfjs-vis 的 fitCallbacks 在每個週期結束（onEpochEnd）時繪製 loss 曲線（S26「Training Function」）。

10.1.10 測試模型：反正規化與繪製結果

模型訓練後，必須測試與評估——方法是檢視模型對一系列不同輸入的預測。但在此之前，必須先把數據**反正規化（un-normalize）**（S26「Test the Model」）：

S26：在 0–1 之間取 100 個等距 x，預測後反正規化

```
let unX = tf.linspace(0, 1, 100);
let unY = model.predict(unX.reshape([100, 1]));

const unNormunX = unX.mul(inputMax.sub(inputMin)).add(inputMin);
const unNormunY = unY.mul(labelMax.sub(labelMin)).add(labelMin);

unX = unNormunX.dataSync();
unY = unNormunY.dataSync();
```

$$\text{原始值} = \text{正規化值} \times (\max - \min) + \min$$

S26：把原始數據與預測直線一起繪出

```
const predicted = Array.from(unX).map((val, i) => {
 return {x: val, y: unY[i]}
});

// Plot the Result
tfPlot([values, predicted], surface1)
```

> **【補充說明（非來源內容）】** `tf.linspace(0, 1, 100)` 產生 100 個介乎 0 與 1 的等距數值（即正規化後的馬力範圍）；`reshape([100, 1])` 把它變成模型期望的 100 列 × 1 欄形狀。tfPlot 的 series 名稱 ['Original','Predicted'] 正好對應 [values, predicted] 兩個系列。

10.2 範例二：以房間數預測樓價（房屋數據）

範例二使用與範例一**相同的原始碼**，但因為使用另一個數據集，程式必須收集不同的數據（S28 開首）。

10.2.1 數據收集

範例二的數據是一個房屋物件清單，數據集是儲存於 https://github.com/meetnandu05/ml1/blob/master/house.json 的 JSON 檔案（S28「Data Collection」）：

S28：房屋數據物件（節錄）

```
{
 "Avg. Area Income": 79545.45857,
 "Avg. Area House Age": 5.682861322,
 "Avg. AreaNumberofRooms": 7.009188143,
```

```
 "Avg. Area Number of Bedrooms": 4.09,
 "Area Population": 23086.8005,
 "Price": 1059033.558,
},
{
 "Avg. Area Income": 79248.64245,
 "Avg. Area House Age": 6.002899808,
 "Avg. AreaNumberofRooms": 6.730821019,
 "Avg. Area Number of Bedrooms": 3.09,
 "Area Population": 40173.07217,
 "Price": 1505890.915,
},
```

10.2.2 與範例一的差異

項目	範例一（S24–S26）	範例二（S28–S30）
數據集	carsData.json（汽車）	house.json（房屋）
x（特徵）	Horsepower（馬力）	房間數（rooms）——S29「Shapes and Units」註明 x = rooms
y（標籤）	Miles_per_Gallon（每加侖英里數）	Price（樓價）
tfPlot 軸標籤	xLabel: 'Horsepower'，yLabel: 'MPG'	xLabel: 'Rooms'，yLabel: 'Price'
訓練週期 epochs	50	**100**
批次大小 batchSize	25	25
模型、編譯、正規化、洗牌、測試	如 10.1 節	完全相同

【來源問題／不一致之處】 S28「Remove Data」的 extractData 函數**仍然**寫作 `return {x:obj.Horsepower, y:obj.Miles_per_Gallon};`，「Fetching Data」也仍然 `fetch("cardata.json")`，文字說明亦照抄範例一的「Horsepower 與 Miles_per_Gallon」。這與房屋數據集的欄位不符，顯然是複製範例一時未經修改。按 S29 的說明（x = rooms）與 S28 的軸標籤（Rooms／Price），正確的函數應類似：`return {x:obj["Avg. AreaNumberofRooms"], y:obj.Price};`，並擷取 house.json。此修正屬講義編者的合理推斷，原教程並未提供。

10.3 兩個範例的完整流程總覽

階段	關鍵 API／函數	對應概念
1. 收集數據	`fetch()`、`.json()`	S14 數據收集（項目工作的 80%）

階段	關鍵 API／函數	對應概念
2. 準備數據	`map(extractData)`、`filter(removeErrors)`、`tf.util.shuffle()`、`tf.tensor2d()`、min-max 正規化	清理、洗牌、張量化、正規化
3. 建立與編譯模型	`tf.sequential()`、`tf.layers.dense()`、`model.compile()`	序列模型、密集層、MSE 損失、SGD 優化器
4. 訓練模型	`model.fit()`、`tfvis.show.fitCallbacks()`	epochs、batchSize、loss 曲線（第 7 節的梯度下降）
5. 評估模型	`tf.linspace()`、`model.predict()`、反正規化、`dataSync()`、`tfvis.render.scatterplot()`	以未見過的輸入檢視預測（第 6.7 節的測試原則）

11. JavaScript 圖表與視覺化程式庫

可同時用於人工智能圖表與其他圖表的 JavaScript 程式庫有：Canvas、Plotly.js、Chart.js、Google Chart、D3.js（S31「Graphic Libraries」；S04「Plotting in the Browser」）。S31 的概覽：

程式庫	教程概覽（S31）
Plotly.js	附有超過 40 種圖表類型、3D 圖表、統計圖與 SVG 地圖的圖表程式庫。
Chart.js	附有多種內置圖表類型：散點（Scatter）、線（Line）、棒（Bar）、雷達（Radar）、圓形與環形（Pie and Doughnut）、極區（Polar Area）、泡泡（Bubble）。
Google Chart	從簡單線形圖到複雜樹狀圖，提供多種內置圖表類型：散點、線、棒／柱、面積、圓形、環形、組織圖、地圖／地理圖。

11.1 HTML Canvas 與自製繪圖器

HTML Canvas 非常適合散點圖、線形圖，以及散點與直線的組合（S32 開首）。

11.1.1 直接在 Canvas 上繪圖

S32「Scatter Plots」：把數據按比例縮放到 400×400 的畫布

```
const xArray = [50,60,70,80,90,100,110,120,130,140,150];
const yArray = [7,8,8,9,9,9,10,11,14,14,15];

// Plot Scatter
ctx.fillStyle = "red";
for (let i = 0; i < xArray.length-1; i++) {
  let x = xArray[i]*400/150;
  let y = yArray[i]*400/15;
  ctx.beginPath();
  ctx.ellipse(x, y, 2, 3, 0, 0, Math.PI * 2);
  ctx.fill();
}
```

S32「Line Graphs」：畫一條斜率 1.2、截距 70 的直線

```
const xMax = canvas.height = canvas.width;
const slope = 1.2;
const intercept = 70;

// Plot Line
ctx.beginPath();
ctx.moveTo(0, intercept);
ctx.lineTo(xMax, xMax * slope + intercept);
ctx.stroke();
```

「Combined」範例把以上兩段合併（先畫散點，再畫直線），其中 `xMax = canvas.height`、`yMax = canvas.width`，散點座標以 `xArray[i] * xMax/150` 與 `yArray[i] * yMax/15` 縮放（S32「Combined」）。

> **【來源問題／不一致之處】** 散點迴圈的條件是 `i < xArray.length-1`，因此最後一個數據點 (150, 15) 不會被畫出；這是原始程式碼的小瑕疵，一般應寫 `i < xArray.length`。另外 Canvas 的 y 軸預設向下，所以未經變換時圖形會上下顛倒——這正是下面 transformXY 方法要解決的問題。

11.1.2 建立繪圖器物件（XYPlotter）

研究人工智能時擁有一個繪圖器物件很有用：讓 AI 更有趣、更視覺化、更易理解（S32）。教程逐步建立 XYPlotter 物件並加入三個方法：

S32「Put the Code in a Library」：完整的 XYPlotter（儲存為 myplotlib.js）

```
function XYPlotter(id) {
this.canvas = document.getElementById(id);
this.ctx = this.canvas.getContext("2d");
this.xMin = 0;
this.yMin = 0;
this.xMax = this.canvas.width;
this.yMax = this.canvas.height;

// Plot Line Function
this.plotLine = function(x0, y0, x, y, color) {
  this.ctx.moveTo(x0, y0);
  this.ctx.lineTo(x, y);
  this.ctx.strokeStyle = color;
  this.ctx.stroke();
}

// Transform XY Function
this.transformXY = function() {
  this.ctx.transform(1, 0, 0, -1, 0, this.canvas.height)
}

// Pot Points Function
this.plotPoints = function(n, xArr, yArr, color, radius = 3) {
  for (let i = 0; i < n; i++) {
    this.ctx.fillStyle = color;
    this.ctx.beginPath();
    this.ctx.ellipse(xArr[i], yArr[i], radius, radius, 0, 0, Math.PI * 2);
    this.ctx.fill();
  }
}

} // End Plotter Object
```

方法	作用（S32）
`plotLine(x0, y0, x, y, color)`	由 (x0, y0) 畫一條直線到 (x, y)，顏色為 color。
`transformXY()`	以 `ctx.transform(1, 0, 0, -1, 0, canvas.height)` 把 y 軸翻轉，令原點移到左下角，座標方向與數學慣例一致。
`plotPoints(n, xArr, yArr, color, radius = 3)`	繪製 n 個點，預設半徑 3。

S32「Plot Some Random Points」

```
// Create a Plotter
let myPlotter = new XYPlotter("myCanvas");

// Create random XY Points
numPoints = 500;
const xPoints = Array(numPoints).fill(0).map(function(){return Math.random() * myPlotter.xMax});
const yPoints = Array(numPoints).fill(0).map(function(){return Math.random() * myPlotter.yMax});

// Plot the Points
myPlotter.plotPoints(numPoints, xPoints, yPoints, "blue");
```

把程式庫儲存為 myplotlib.js 後，以 `<script src="myplotlib.js"></script>` 加入 HTML 頁面即可使用（S32「Use it in Your HTML Pages」）。

> **【來源問題／不一致之處】** 第 6.5 與 6.6 節的感知器程式呼叫了 `plotter.plotPoint(x, y, color)`（單數），但 XYPlotter 程式庫只定義了 `plotPoints`（複數，需傳入數量與陣列）。兩者名稱與簽名不一致，是教程各頁之間的不一致之處；實作時需自行補上單點版本，例如 `this.plotPoint = function(x, y, color, radius = 3) {...}`。

11.2 Plotly.js

Plotly.js 的基本模式是：定義 data（陣列，每個元素是一個 trace）、定義 layout、呼叫 `Plotly.newPlot("myPlot", data, layout)`。S33 的範例一覽：

圖表	關鍵設定（S33）
散點圖	`mode:"markers", type:"scatter"`；layout 設定 xaxis／yaxis 的 range 與 title（Square Meters／Price in Millions）及標題 House Prices vs. Size
線形圖	同上，但 `mode: "lines"`
線性圖（由運算式產生）	`let exp = "x + 17";` 以 `eval(exp)` 在 x = 0…10 產生 y 值；layout 標題 "y = " + exp
多條直線	三個運算式 "x"、"1.5*x"、"1.5*x + 7"，各自產生 x／y 陣列，data 含三個 trace
棒形圖	`type: "bar"`；x 為國家 ["Italy","France","Spain","USA","Argentina"]，y 為 [55, 49, 44, 24, 15]，標題 World Wide Wine Production

圖表	關鍵設定（S33）
水平棒形圖	交換 x／y，並加 `orientation: "h"`
圓形圖	把 x／y 改為 `labels`／`values`，`type: "pie"`
環形圖	圓形圖加上 `hole: .4`
繪製方程	`let exp = "Math.sin(x)";` 以 0.1 為步長在 0…10 產生點，`mode:"lines"`

S33「Multiple Lines」：比較 y = x、y = 1.5x、y = 1.5x + 7（斜率與截距的視覺化）

```
let exp1 = "x";
let exp2 = "1.5*x";
let exp3 = "1.5*x + 7";

// Generate values
const x1Values = []; const x2Values = []; const x3Values = [];
const y1Values = []; const y2Values = []; const y3Values = [];
for (let x = 0; x <= 10; x += 1) {
  x1Values.push(x); x2Values.push(x); x3Values.push(x);
  y1Values.push(eval(exp1));
  y2Values.push(eval(exp2));
  y3Values.push(eval(exp3));
}

// Define Data
const data = [
  {x: x1Values, y: y1Values, mode:"lines"},
  {x: x2Values, y: y2Values, mode:"lines"},
  {x: x3Values, y: y3Values, mode:"lines"}
];

// Define Layout
const layout = {title: "[y=" + exp1 + "] [y=" + exp2 + "] [y=" + exp3 + "]"};

// Display using Plotly
Plotly.newPlot("myPlot", data, layout);
```

S33：棒形圖、圓形圖與環形圖

```
const xArray = ["Italy","France","Spain","USA","Argentina"];
const yArray = [55, 49, 44, 24, 15];

const data = [{
  x: xArray,
  y: yArray,
  type: "bar"
}];
const layout = {title:"World Wide Wine Production"};
Plotly.newPlot("myPlot", data, layout);

// 圓形圖：改用 labels / values
```

```
const pieData = [{labels: xArray, values: yArray, type: "pie"}];
// 環形圖：加 hole
const donutData = [{labels: xArray, values: yArray, hole: .4, type: "pie"}];
```

> **【補充說明（非來源內容）】** 教程以 `eval()` 把字串運算式轉為數值，方便示範；在正式程式中 eval 有安全與效能風險，應改用函數或數學運算式解析器（例如 8.2.1 節提到的 Math.js）。

11.3 Chart.js

Chart.js 是製作 HTML 圖表的免費 JavaScript 程式庫，是最簡單的 JavaScript 視覺化程式庫之一，內置：散點圖、線形圖、棒形圖、圓形圖、環形圖、泡泡圖、面積圖、雷達圖、混合圖（S34 開首）。

11.3.1 如何使用

1. 加入 CDN 連結：`<script src="https://cdnjs.cloudflare.com/ajax/libs/Chart.js/2.9.4/Chart.js"></script>`
2. 加入一個 `<canvas id="myChart" style="width:100%;max-width:700px"></canvas>`（canvas 必須有唯一的 id）
3. 以 `new Chart("myChart", {type: "scatter" | "line" | "bar", data: {}, options: {}})` 建立圖表。

11.3.2 範例一覽

S34「Scatter Plots」：數據以 {x, y} 物件陣列提供

```
const xyValues = [
  {x:50, y:7}, {x:60, y:8}, {x:70, y:8}, {x:80, y:9}, {x:90, y:9}, {x:100, y:9},
  {x:110, y:10}, {x:120, y:11}, {x:130, y:14}, {x:140, y:14}, {x:150, y:15}
];

new Chart("myChart", {
  type: "scatter",
  data: {
    datasets: [{
      pointRadius: 4,
      pointBackgroundColor: "rgba(0,0,255,1)",
      data: xyValues
    }]
  },
  options:{...}
});
```

S34「Line Graphs」：labels 為 x，datasets.data 為 y

```
const xValues = [50,60,70,80,90,100,110,120,130,140,150];
const yValues = [7,8,8,9,9,9,10,11,14,14,15];
```

```
new Chart("myChart", {
  type: "line",
  data: {
    labels: xValues,
    datasets: [{
      backgroundColor:"rgba(0,0,255,1.0)",
      borderColor: "rgba(0,0,255,0.1)",
      data: yValues
    }]
  },
  options:{...}
});
// 把 borderColor 設為 "rgba(0,0,0,0)"（透明）即可把線形圖變成散點效果
```

範例	關鍵設定（S34）
多條線	labels 為 [100,200,⋯,1000]；三個 dataset 分別以 borderColor red／green／blue，`fill: false`；options `legend: {display: false}`
線性圖	自訂 `generateData("x * 2 + 7", 0, 10, 0.5)` 以 eval 產生點；dataset 設 `fill: false, pointRadius: 1, borderColor: "rgba(255,0,0,0.5)"`
函數圖	與線性圖相同，只改參數：`generateData("Math.sin(x)", 0, 10, 0.5)`
棒形圖	`type: "bar"`；labels 為五個國家，data 為 [55, 49, 44, 24, 15]，`backgroundColor: barColors`
棒形圖上色	只為一條上色 `barColors = ["blue"]`；全部同色 `barColors = "red"`；色階 `["rgba(0,0,255,1.0)", "rgba(0,0,255,0.8)", ... 0.2]`
水平棒形圖	把 type 改為 `"horizontalBar"`
圓形圖	`type: "pie"`，options 設 `title: {display: true, text: "World Wide Wine Production"}`
環形圖	把 type 改為 `"doughnut"`

S34「Linear Graphs」：generateData 輔助函數

```
const xValues = [];
const yValues = [];
generateData("x * 2 + 7", 0, 10, 0.5);

new Chart("myChart", {
  type: "line",
  data: {
    labels: xValues,
    datasets: [{
      fill: false,
      pointRadius: 1,
      borderColor: "rgba(255,0,0,0.5)",
      data: yValues
    }]
  },
```

```
  options: {...}
});

function generateData(value, i1, i2, step = 1) {
  for (let x = i1; x <= i2; x += step) {
    yValues.push(eval(value));
    xValues.push(x);
  }
}
```

> **【補充說明（非來源內容）】** 教程使用的是 Chart.js **2.9.4** 版。在 Chart.js 3.x／4.x 中，`horizontalBar` 類型已被移除（改用 `type: "bar"` 加 `options.indexAxis: 'y'`），而 `legend` 與 `title` 選項亦移到 `options.plugins` 之下。若使用新版本，教程程式碼需相應修改（此為講義編者的補充提示）。

11.4 Google Chart

Google Chart 圖庫提供大量可即用的圖表類型，從簡單線形圖到複雜的層次樹狀圖（S35 開首）。使用步驟（S35「How to Use Google Chart?」）：

1. 加入圖表載入器：`<script src="https://www.gstatic.com/charts/loader.js"></script>`
2. 加入一個顯示圖表的 `<div id="myChart" style="max-width:700px; height:400px"></div>`（必須有唯一 id）
3. 載入 Google Graph API：載入 Visualization API 與 corechart 套件
   `google.charts.load('current',{packages:['corechart']});`，並設定 API 載入完成後的回呼
   `google.charts.setOnLoadCallback(drawChart);`

S35「Line Graph」

```
function drawChart() {

// Set Data
const data = google.visualization.arrayToDataTable([
  ['Price', 'Size'],
  [50,7],[60,8],[70,8],[80,9],[90,9],[100,9],
  [110,10],[120,11],[130,14],[140,14],[150,15]
  ]);

// Set Options
const options = {
  title: 'House Prices vs Size',
  hAxis: {title: 'Square Meters'},
  vAxis: {title: 'Price in Millions'},
  legend: 'none'
};

// Draw Chart
const chart = new google.visualization.LineChart(document.getElementById('myChart'));
```

```
chart.draw(data, options);
}
```

圖表	改動（S35）
散點圖	把 `google.visualization.LineChart` 改為 `google.visualization.ScatterChart`
棒形圖	數據為 [['Contry', 'Mhl'], ['Italy', 55], ['France', 49], ['Spain', 44], ['USA', 24], ['Argentina', 15]]；使用 `google.visualization.BarChart`；options 標題 World Wide Wine Production
圓形圖	把 `BarChart` 換成 `PieChart`
3D 圓形圖	在 options 加入 `is3D: true`

【來源問題／不一致之處】 S35「Scatter Plots」的文字說要改為 ScatterChart，但隨附的程式碼片段仍然寫着 `new google.visualization.LineChart(...)`，程式碼與說明不符。另外數據表頭 'Contry' 應為 'Country'，且線形圖的表頭把第一欄標為 'Price'、第二欄標為 'Size'，而實際數值次序是（面積, 價格），表頭標籤與數據倒置。

11.5 D3.js

D3.js 是**根據數據操作 HTML** 的 JavaScript 程式庫（S36 開首）。使用方法：加入 `<script src="//d3js.org/d3.v3.min.js"></script>`。以下一行選取 body 元素並附加一段文字為 “Hello World!” 的段落（S36「How to Use D3.js?」）：

S36：D3 入門

```
d3.select("body").append("p").text("Hello World!");
```

S36「Scatter Plot」：以 SVG 繪製 100 個隨機點並加上座標軸

```
// Set Dimensions
const xSize = 500;
const ySize = 500;
const margin = 40;
const xMax = xSize - margin*2;
const yMax = ySize - margin*2;

// Create Random Points
const numPoints = 100;
const data = [];
for (let i = 0; i < numPoints; i++) {
  data.push([Math.random() * xMax, Math.random() * yMax]);
}

// Append SVG Object to the Page
```

```
const svg = d3.select("#myPlot")
  .append("svg")
  .append("g")
  .attr("transform","translate(" + margin + "," + margin + ")");

// X Axis
const x = d3.scaleLinear()
  .domain([0, 500])
  .range([0, xMax]);
svg.append("g")
  .attr("transform", "translate(0," + yMax + ")")
  .call(d3.axisBottom(x));

// Y Axis
const y = d3.scaleLinear()
  .domain([0, 500])
  .range([ yMax, 0]);
svg.append("g")
  .call(d3.axisLeft(y));

// Dots
svg.append('g')
  .selectAll("dot")
  .data(data).enter()
  .append("circle")
  .attr("cx", function (d) { return d[0] } )
  .attr("cy", function (d) { return d[1] } )
  .attr("r", 3)
  .style("fill", "Red");
```

這段程式的結構：設定尺寸與邊距 → 產生 100 個隨機點 → 在 #myPlot 內附加 SVG 與 g 群組並平移邊距 → 以 `d3.scaleLinear()` 建立 x、y 比例尺並加上 `axisBottom`／`axisLeft` 座標軸 → 以 data-join（`.data(data).enter().append("circle")`）為每個數據點建立一個半徑 3 的紅色圓點。

> **【來源問題／不一致之處】** S36 引入的是 **d3.v3**，但程式碼使用 `d3.scaleLinear()`、`d3.axisBottom()`、`d3.axisLeft()`——這些是 D3 **v4 及以後**的 API（v3 對應為 `d3.scale.linear()` 與 `d3.svg.axis()`）。因此按原文直接執行可能出錯；實作時宜改用較新版本（例如 `https://d3js.org/d3.v7.min.js`）。此為講義編者補充。

11.6 五種繪圖方式的比較

程式庫	引入方式	繪圖目標元素	特點（綜合 S31–S36）
HTML Canvas	瀏覽器原生，無需引入	`<canvas>`	完全自主控制；需自行處理座標縮放與 y 軸翻轉；教程以此建立 XYPlotter 並用於感知器視覺化

程式庫	引入方式	繪圖目標元素	特點（綜合 S31–S36）
Plotly.js	CDN script	任何 `<div id>`	40+ 圖表類型、3D、統計圖、SVG 地圖；data／layout 物件式 API，最少程式碼
Chart.js	CDN script（2.9.4）	`<canvas id>`	簡單易用；以 type／data／options 設定；多種內置圖表
Google Chart	charts loader.js	`<div id>`	需先 load 套件並設定回呼；DataTable 式數據；可 3D
D3.js	d3js.org script	SVG（透過 d3.select）	最靈活、最底層；以數據驅動 DOM／SVG；需較多程式碼

12. 來源問題與不一致之處總覽

以下彙整講義各節以【來源問題／不一致之處】標示的項目，方便讀者一次過查閱。這些問題全部來自 W3Schools 原文本身（排版筆誤、程式碼錯漏、前後不一致，或與其他可靠來源不符），講義在正文中已按語意補正並註明。

#	位置（來源）	問題	講義處理
1	整體	指示提及「附件資料」，但工作區沒有任何附件。	以 W3Schools AI 網域為唯一主要來源；見 0.2 節。
2	S02 / S17	Arthur Samuel 1959 年引文在兩頁措辭不同（subfield of computer science vs field of study）。	分別保留；1.1 節註明。
3	S17	Tom Mitchell 的學習定義標註為 1999 年；該定義出自其 1997 年教科書。	1.8 節註明並引外部來源。
4	S37	引文作者拼作 Yampolsky（學者常見拼法為 Yampolskiy），且無文獻出處。	按原文保留；2.1 節註明。
5	S38	書面語言年表次序非嚴格按年份；以 AC 表示公元後。	保留原次序；2.2 節註明。
6	S39	「一小時 | 60° | 1 × 60 = 60 minutes」把小時與度數並列；60 的因數清單一處漏了 20。	改為 60 分鐘；2.3 節註明。
7	S40	iPhone 12 每秒 110 億條指令 vs 表中 2020 年 100 億；2020 年一行無電腦名稱；千分位用歐陸格式。	照錄並註明；改用逗號千分位。
8	S42	年表把 Sophia 拼作 Sofia；電影 D.A.R.Y.L. 寫作 D.A.R.Y.。	統一為 Sophia／D.A.R.Y.L.。
9	S43	「47% 的所有職業」——原研究為「約 47% 的美國總就業人數處於高風險」（702 種職業）。	2.8 節補充原研究表述。
10	S45 / S47 / S50	Scalar／Vector／Matrix／Tensor 示意表格為巢狀 HTML，純文字無法還原。	3.4 節以新表重新整理。
11	S49	玫瑰例子的 mB 矩陣程式碼缺少一個右方括號。	3.6.6 節補正。
12	S53	方差程式碼項數與數據不符（四個 9、一個 14）；標準差程式碼平均數漏了一個 14；math.js std 範例數據 [7,8,8,9,9,9,9,10,11,14,15] 與表格不符。	4.3 節全部修正並給出正確數值（方差 ≈ 6.9587，標準差 ≈ 2.638）。
13	S16	「Linear Graphs」範例的 yArray 有 12 個值（多一個 9），xArray 只有 11 個。	5.6.1 節註明。
14	S08	組成部分小節編號次序為 1,2,3,4,6,5；threshold 多處拼作 treshold。	6.3 節重新編號 1–7。
15	S09 / S10 / S11 vs S32	感知器程式呼叫 plotter.plotPoint(x, y, color)，但 XYPlotter 程式庫只定義 plotPoints(n, xArr, yArr, color, radius)。	11.1.2 節註明，建議自行補上單點方法。
16	S18	Brain.js 第一個範例第三、四個訓練物件缺少右大括號；深灰註釋寫 64 而程式與表格用 65。	8.3 節補正並註明。

#	位置（來源）	問題	講義處理
17	S19	TensorFlow.js 發佈日期寫作 2019 年 1 月；其他來源為 2018 年 3 月宣佈、2019 年 3 月發佈 1.0。	9.1 節註明並引外部來源。
18	S21	說預測 10 個 y 值，迴圈實際產生 11 個（x = 0…10）。	9.4.2 節註明。
19	S22	棒形圖範例兩個數據點的 index 都是 2。	9.5 節註明。
20	S28	範例二的 extractData 仍用 Horsepower／Miles_per_Gallon，fetch 仍讀 cardata.json，文字說明照抄範例一。	10.2.2 節註明並提出合理修正。
21	S32	Canvas 散點迴圈 i < xArray.length-1 漏畫最後一點。	11.1.1 節註明。
22	S35	散點圖說明要改用 ScatterChart 但程式碼仍是 LineChart；'Contry' 拼錯；線形圖表頭 Price／Size 與數據次序倒置。	11.4 節註明。
23	S36	引入 d3.v3 但程式碼使用 v4+ 的 API（scaleLinear、axisBottom、axisLeft）。	11.5 節註明，建議改用新版本。
24	S12	Trainer 程式碼中 total 未宣告（全域變數）；this.cost; 一行無實際作用。	7.6 節註明。

13. 重點總結

以下按學習順序概括本講義的關鍵要點：

13.1 概念層面

- **AI 的層級**：人工智能 ⊃ 機器學習 ⊃ 深度學習；神經網絡是機器學習的方法，深度學習以多層神經網絡處理大量數據。窄 AI（弱 AI）模擬人類智能並限於特定範疇；強 AI 旨在複製人類智能，仍屬理論階段。
- **機器學習的本質**：傳統編程是「數據 + 演算法 = 結果」，機器學習是「數據 + 結果 = 演算法」；Mitchell 的 E–T–P 定義把「學習」界定為表現隨經驗改善。
- **歷史視角**：AI 是人類對數字與計算需求的延伸——從巴比倫六十進制、算盤、Babbage 的機械計算機、Zuse Z3、真空管／晶體管／集成電路，到 1950 年代 Turing 與 McCarthy 開創 AI，再到 2010 年前後因算力、儲存、數據與方法同時成熟而爆發的深度學習革命。
- **心智與職業**：AI 由反應式機器走向機器學習，未來目標是心智理論與自我意識；Frey 與 Osborne（2013）估計約 47% 的美國就業處於高電腦化風險。

13.2 數學與統計工具

- **線性函數 y = ax + b**：a 是斜率、b 是截距；機器學習寫作 y = b + wx（權重與偏置）。

- **線性代數物件**：純量（0 維）、向量（1 維）、矩陣（2 維）、張量（N 維）；矩陣加減需同維度，矩陣乘法需 A 的欄數等於 B 的列數並以點積計算；單位矩陣相當於 1。
- **描述統計**：平均數、中位數、眾數描述集中趨勢；方差與標準差描述離散程度；離群值會大幅影響平均數。
- **正態分佈**：68.27%／95.45%／99.73% 的觀察值分別落在 1／2／3 個標準差之內；誤差幅度量化抽樣的不確定性。
- **概率** = 事件可以發生的方式數 ÷ 所有可能結果數，取值 0 至 1。

13.3 機器學習實務

- **數據先行**：項目高達 80% 的工作是收集、清理與準備數據；隨機抽樣避免抽樣偏差；訓練前必須洗牌並正規化。
- **術語**：標籤（要預測的 y）、特徵（輸入 x）、模型（y 與 x 的關係）、訓練（由數據計算參數）、推論（用模型預測新數據）。
- **學習類型**：監督式（有標籤，用於分類與預測）、非監督式（無標籤，找模式／聚類）、強化（依回饋改進）、自監督。
- **感知器**：輸入 × 權重 → 加總 → 與閾值比較（啟動函數）→ 二進制輸出；加入偏置輸入 1；以「error = desired − guess」按學習率調整權重；只能處理線性可分問題，多層網絡可突破此限制。
- **訓練迴圈**：成本函數（均方誤差）量度直線貼合程度；梯度下降沿偏導數的反方向逐步更新權重與偏置，直至成本不再下降。
- **評估**：必須以未見過的數據測試；可透過調整學習率、增加數據量與迭代次數來調校。

13.4 工具與實作

- **語言**：LISP、R、Python、C++、Java、JavaScript、SQL 各有角色；JavaScript 的優勢是在瀏覽器執行、無需安裝、私隱較佳、可用 WebGL 硬件加速。
- **程式庫**：Brain.js（隱藏數學、幾行即可訓練）、ml5.js（三行圖像分類）、Math.js（矩陣與統計運算）、TensorFlow.js（張量、模型、優化器）、tfjs-vis（視覺化）。
- **TensorFlow.js 工作流程**：收集數據 → 建立模型（tf.sequential）→ 加入層（tf.layers.dense）→ 編譯（meanSquaredError + sgd）→ 訓練（model.fit，epochs／batchSize）→ 使用（model.predict）。
- **圖表**：Canvas（自製 XYPlotter）、Plotly.js、Chart.js、Google Chart、D3.js 都能繪製散點圖與線形圖——視覺化是理解 AI 的重要輔助。

14. 資料來源清單

14.1 主要來源：W3Schools AI／Machine Learning 教程（擷取日期 2026-10-14）

基底網址：https://www.w3schools.com/ai/ 。下表的「頁面」欄為導覽列名稱（括號內為頁面主標題），網址欄為基底網址後的檔名。

編號	頁面（導覽名稱／頁面標題）	網址
S01	ML Intro（Machine Learning）	https://www.w3schools.com/ai/default.asp
S02	ML and AI（What is AI? / Artificial Intelligence）	https://www.w3schools.com/ai/ai_whatis.asp
S03	ML Languages（Machine Learning Languages）	https://www.w3schools.com/ai/ai_languages.asp
S04	ML JavaScript（Machine Learning in JavaScript）	https://www.w3schools.com/ai/ai_javascript.asp
S05	ML Examples（Machine Learning Examples）	https://www.w3schools.com/ai/ai_examples.asp
S06	ML Linear Graphs（Linear Graphs）	https://www.w3schools.com/ai/ai_linear_graphs.asp
S07	ML Scatter Plots（Scatter Plots）	https://www.w3schools.com/ai/ai_scatter_plots.asp
S08	ML Perceptrons（Perceptrons）	https://www.w3schools.com/ai/ai_perceptrons.asp
S09	ML Recognition（Pattern Recognition）	https://www.w3schools.com/ai/ai_recognition.asp
S10	ML Training（Training a Perceptron）	https://www.w3schools.com/ai/ai_training.asp
S11	ML Testing（Testing a Perceptron）	https://www.w3schools.com/ai/ai_perceptron_testing.asp
S12	ML Learning（Machine Learning – Learning is Looping / Gradient Descent）	https://www.w3schools.com/ai/ai_learning.asp
S13	ML Terminology（ML Terminology）	https://www.w3schools.com/ai/ai_ml_terminology.asp
S14	ML Data（Machine Learning Data）	https://www.w3schools.com/ai/ai_data.asp
S15	ML Clustering（Data Clusters）	https://www.w3schools.com/ai/ai_clustering.asp
S16	ML Regressions（Linear Regressions）	https://www.w3schools.com/ai/ai_regressions.asp
S17	ML Deep Learning（Deep Learning (DL)）	https://www.w3schools.com/ai/ai_neural_networks.asp
S18	ML Brain.js（Brain.js）	https://www.w3schools.com/ai/ai_brainjs.asp
S19	TFJS Tutorial（TensorFlow.js Tutorial）	https://www.w3schools.com/ai/ai_tensorflow_intro.asp
S20	TFJS Operations（TensorFlow Operations）	https://www.w3schools.com/ai/ai_tensorflow_operations.asp
S21	TFJS Models（TensorFlow Models）	https://www.w3schools.com/ai/ai_tensorflow_model.asp

編號	頁面（導覽名稱／頁面標題）	網址
S22	TFJS Visor（TensorFlow.js Visor）	https://www.w3schools.com/ai/ai_tensorflow_visor.asp
S23	Ex1 Intro（TensorFlow Example 1）	https://www.w3schools.com/ai/ai_tensorflow_example_2d.asp
S24	Ex1 Data（Example 1 Data）	https://www.w3schools.com/ai/ai_tensorflow_data.asp
S25	Ex1 Model（Example 1 Model）	https://www.w3schools.com/ai/ai_tensorflow_modelling.asp
S26	Ex1 Training（Example 1 Training）	https://www.w3schools.com/ai/ai_tensorflow_training.asp
S27	Ex2 Intro（TensorFlow Example 2）	https://www.w3schools.com/ai/ai_tensorflow_example_houses.asp
S28	Ex2 Data（Example 2 Data）	https://www.w3schools.com/ai/ai_tensorflow_ex2_data.asp
S29	Ex2 Model（Example 2 Model）	https://www.w3schools.com/ai/ai_tensorflow_ex2_model.asp
S30	Ex2 Training（Example 2 Training）	https://www.w3schools.com/ai/ai_tensorflow_ex2_training.asp
S31	Graph Intro（JavaScript Graphics）	https://www.w3schools.com/ai/ai_graphics.asp
S32	Graph Canvas（HTML Canvas）	https://www.w3schools.com/ai/ai_canvas.asp
S33	Graph Plotly.js（Plotly.js）	https://www.w3schools.com/ai/ai_plotly.asp
S34	Graph Chart.js（Chart.js）	https://www.w3schools.com/ai/ai_chartjs.asp
S35	Graph Google（Google Chart）	https://www.w3schools.com/ai/ai_google_chart.asp
S36	Graph D3.js（D3.js）	https://www.w3schools.com/ai/ai_d3js.asp
S37	History of Intelligence（Human Intelligence）	https://www.w3schools.com/ai/ai_history_brain.asp
S38	History of Languages（The History of Languages）	https://www.w3schools.com/ai/ai_history_languages.asp
S39	History of Numbers（The History of Numbers）	https://www.w3schools.com/ai/ai_history_numbers.asp
S40	History of Computing（The History of Computing）	https://www.w3schools.com/ai/ai_history_computers.asp
S41	History of Robots（History of Robots）	https://www.w3schools.com/ai/ai_robots.asp
S42	History of AI（The History of AI）	https://www.w3schools.com/ai/ai_history.asp
S43	Job Replacements（Job Replacement）	https://www.w3schools.com/ai/ai_job_replacement.asp
S44	Theory of Mind（Theory of Mind）	https://www.w3schools.com/ai/ai_mind.asp
S45	Mathematics（ML Mathematics）	https://www.w3schools.com/ai/ai_mathematics.asp
S46	Linear Functions（Linear Functions）	https://www.w3schools.com/ai/ai_linear.asp

編號	頁面（導覽名稱／頁面標題）	網址
S47	Linear Algebra（Linear Algebra）	https://www.w3schools.com/ai/ai_algebra.asp
S48	Vectors（Vectors）	https://www.w3schools.com/ai/ai_vectors.asp
S49	Matrices（Matrices）	https://www.w3schools.com/ai/ai_matrices.asp
S50	Tensors（Tensors）	https://www.w3schools.com/ai/ai_tensors.asp
S51	Statistics（Machine Learning Statistics）	https://www.w3schools.com/ai/ai_statistics.asp
S52	Descriptive（Descriptive Statistics）	https://www.w3schools.com/ai/ai_statistics_descriptive.asp
S53	Variability（Statistic Variability (Spread)）	https://www.w3schools.com/ai/ai_variance.asp
S54	Distribution（Distribution）	https://www.w3schools.com/ai/ai_distribution.asp
S55	Probability（Probability）	https://www.w3schools.com/ai/ai_probability.asp

14.2 補充來源（僅用於【補充說明】與【來源問題】方框中的核對）

用途	來源
TensorFlow.js 的發佈時間（9.1 節）	Wikipedia，“TensorFlow” — https://en.wikipedia.org/wiki/TensorFlow （記載 2018 年 3 月宣佈 TensorFlow.js）；Medium／AI Lab Telkom University，“Learn and Play with TensorFlow.js [Part 0]” — https://medium.com/ailab-telu/learn-and-play-with-tensorflow-js-part-0-introduction-and-installation-d4d19e9982ea （記載 2019 年 3 月 7 日發佈 1.0）；DEV Community／IBM Developer，“Machine Learning In Node.js With TensorFlow.js” — https://dev.to/ibmdeveloper/machine-learning-in-nodejs-with-tensorflowjs-1g1p
Tom Mitchell 定義的出處年份（1.8 節）	Wikipedia，“Machine learning” — https://en.wikipedia.org/wiki/Machine_learning ；ApX Machine Learning，“Defining Machine Learning” — https://apxml.com/courses/introduction-to-machine-learning/chapter-1-what-is-machine-learning/defining-machine-learning （均註明 Mitchell, *Machine Learning*, McGraw-Hill, 1997）
Frey 與 Osborne 研究的原始表述（2.8 節）	Oxford Martin School，“The Future of Employment: How susceptible are jobs to computerisation?” — https://www.oxfordmartin.ox.ac.uk/publications/the-future-of-employment （702 種職業；約 47% 美國總就業處於風險）
其他補充說明	凡未另註出處的【補充說明】（例如二進制換算、迴歸係數、方差數值、偏導數推導、Chart.js／D3.js 版本差異）均為講義編者根據教程提供的數據自行計算或根據一般技術常識補充，讀者可自行驗證。

附錄：中英術語對照表

按講義出現順序大致排列；「節」欄指首次或主要出現的章節。

中文（香港用語）	英文	節
人工智能	Artificial Intelligence (AI)	1.1
機器智能／電腦智能	Machine Intelligence / Computer Intelligence	1.1
窄人工智能／弱人工智能	Narrow AI / Weak AI	1.3
強人工智能	Strong AI	1.3
機器學習	Machine Learning (ML)	1.4
神經網絡	Neural Networks (NN)	1.5
感知器	Perceptron	1.6, 6
多層感知器	Multi-Layer Perceptron (MLP)	1.6, 6.4
深度神經網絡	Deep Neural Network (DNN)	1.6
深度學習	Deep Learning (DL)	1.7
輸入層／隱藏層／輸出層	Input / Hidden / Output layer	1.6
大數據	Big Data	5.1
數據挖掘	Data Mining	5.1
數據科學	Data Science	1.2, 8.4
認知智能	Cognitive Intelligence	2.1
抽象思維	Abstract Thinking	2.1
心智理論	Theory of Mind	2.7
認知科學	Cognitive Science	2.7
心智模型	Mental Model	2.7
反應式機器	Reactive Machines	2.7
自我意識／機器意識	Self-Awareness / Machine Consciousness	2.7
六十進制	Sexagesimal (base 60)	2.3
十二進制	Dozenal (base 12)	2.3
二進制數字／數碼數字	Binary / Digital Numbers (base 2)	2.3
位值制	Place-value system	2.3
算盤	Abacus	2.4
差分機／分析機	Difference Engine / Analytical Engine	2.4
繼電器／真空管／晶體管／集成電路	Relays / Vacuum tubes / Transistors / Integrated circuits	2.4
摩爾定律	Moore's Law	2.4
工業機械人	Industrial Robots	2.5
線性函數／線性方程	Linear Function / Linear Equation	3.2
斜率	Slope	3.3
截距	Intercept	3.3
線性迴歸	Linear Regression	3.2, 5.6
多項式迴歸	Polynomial Regression	5.6

中文（香港用語）	英文	節
線性最小二乘法	Linear Least Squares (LLS)	3.2
線性代數	Linear Algebra	3.4
純量	Scalar	3.4
向量	Vector	3.5
大小／方向	Magnitude / Direction	3.5
矩陣	Matrix	3.6
方陣／對角矩陣／純量矩陣／單位矩陣／零矩陣	Square / Diagonal / Scalar / Identity / Zero matrix	3.6
轉置	Transpose	3.6
點積	Dot product	3.6
矩陣分解	Matrix Factorization	3.6
張量	Tensor	3.7, 9.2
秩	Rank	3.7, 9.2
形狀	Shape	9.2
數據類型	dtype (data type)	9.2
重塑	Reshape	9.3
推論統計／描述統計	Inferential / Descriptive Statistics	4.1
總體／樣本／普查	Population / Sample / Census	4.1, 5.1
隨機樣本／抽樣偏差	Random Sample / Sampling Bias	5.1
平均數／中位數／眾數	Mean / Median / Mode	4.2
離群值	Outlier	4.2
方差	Variance	4.3
標準差	Standard Deviation (σ)	4.3
正態分佈／高斯分佈／鐘形曲線	Normal / Gaussian Distribution / Bell Curve	4.4
68–95–99.7 法則／經驗法則	68–95–99.7 Rule / Empirical Rule	4.4
誤差幅度	Margin of Error	4.4
偏度／峰度	Skewness / Kurtosis	4.4
概率	Probability	4.5
定量數據／定性數據	Quantitative / Qualitative data	5.1
散點圖	Scatter Plot	5.2
標籤	Label	5.3
特徵	Feature	5.3
權重	Weight	5.3, 6
偏置	Bias	5.3, 6.6
模型	Model	5.3

中文（香港用語）	英文	節
訓練／推論	Training / Inference	5.3
監督式學習	Supervised Learning	5.4
非監督式學習	Unsupervised Learning	5.4
強化學習	Reinforcement Learning	5.4
自監督學習	Self-Supervised Learning	5.4
聚類	Clustering / Clusters	5.5
相關系數	Correlation Coefficient (r)	5.5
閾值	Threshold	6.2
加權總和	Weighted Sum	6.2
啟動函數	Activation Function	6.3
線性二元分類器	Linear Binary Classifier	6.5
模式識別	Pattern Recognition	6.5
線性可分	Linearly Separable	6.4
學習率	Learning Rate	6.6
反向傳播	Backpropagation	6.6
成本函數／誤差函數／損失函數	Cost / Error / Loss Function	7.4
均方誤差	Mean Squared Error (MSE)	7.4
梯度下降	Gradient Descent	7
隨機梯度下降	Stochastic Gradient Descent (SGD)	9.4
偏導數	Partial Derivative	7.5
迭代／週期	Iteration / Epoch	7.1, 10.1
批次	Batch	10.1
洗牌	Shuffle	10.1
正規化／反正規化	Normalization / Un-normalization	10.1
序列模型	Sequential Model	9.4
密集層	Dense Layer	9.4
優化器	Optimizer	9.4
遞歸函數	Recursive Functions	8.1
自然語言處理	Natural Language Processing (NLP)	1.3
硬件加速	Hardware Acceleration	8.2